



《面向对象程序设计》之——

绪 论

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

授课教师

连盛 Homepage: <http://lancerlian.win/>

个人简介:

连盛, 男, 讲师, 硕导, 校聘副教授, 2022年加入福建省网络计算与智能信息处理重点实验室。研究兴趣包括智能医学图像分析、计算机视觉、人工智能、机器学习等。曾在MedIA、Neural Networks、Brain、CIBM、IEEE TCBB、IEEE TMM、BSPC 以及 MICCAI, BIBM, ICPR 等国际权威期刊/会议上发表了多篇学术论文, 同时也是 TNNLS、MedIA、KBS、PR、MICCAI、ICME 等国际权威期刊/会议的审稿人。教育背景如下:

2015.09~2021.12: 厦门大学, 信息学院, 博士(硕博连读), 导师: 李绍滋教授

2019.12~2021.09: 加拿大韦仕敦大学, 联合培养博士, 导师: 李硕教授

2010.09~2014.06: 华中科技大学, 计算机学院, 本科

招生信息:

目前硕士招生专业: 计算机科学与技术、电子信息等。欢迎勤奋好学, 具有良好编程能力和数学英语能力, 对智能医学图像分析、计算机视觉、人工智能、机器学习感兴趣的研究生和本科生与我联系。

办公地址: 计算机与大数据学院4号楼130。

联系邮箱: shenglian@fzu.edu.cn



成绩计算

- 平时成绩：考勤，作业等， 50% （平时成绩不及格不允许参加课程期末考试）
- 期末考： 50%



期末考试事项: Pintia

- 课程班级
- 作业
- 例题
- 考试形式



课程展示事项

任务方式

- 1~2人一组，最多12组
- 展示时间：待定
- 方式：PPT（+程序展示）
- 每组10~15分钟



推荐选题

- 利用C++生成决策树
- 食堂座位管理系统
- 天气应用（选地址、调用在线API）
- 小游戏
- 其他...

C++实现ID3决策树

- 什么是决策树?
- 信息熵、信息增益?

输入: 训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;

属性集 $A = \{a_1, a_2, \dots, a_d\}$.

过程: 函数 TreeGenerate(D, A)

```
1: 生成结点 node;  
2: if  $D$  中样本全属于同一类别  $C$  then  
3:   将 node 标记为  $C$  类叶结点; return  
4: end if  
5: if  $A = \emptyset$  OR  $D$  中样本在  $A$  上取值相同 then  
6:   将 node 标记为叶结点, 其类别标记为  $D$  中样本数最多的类; return  
7: end if  
8: 从  $A$  中选择最优划分属性  $a_*$ ;  
9: for  $a_*$  的每一个值  $a_*^v$  do  
10:  为 node 生成一个分支; 令  $D_v$  表示  $D$  中在  $a_*$  上取值为  $a_*^v$  的样本子集;  
11:  if  $D_v$  为空 then  
12:    将分支结点标记为叶结点, 其类别标记为  $D$  中样本最多的类; return  
13:  else  
14:    以 TreeGenerate( $D_v, A \setminus \{a_*\}$ ) 为分支结点  
15:  end if  
16: end for
```

输出: 以 node 为根结点的一棵决策树

食堂座位管理系统

- 某校食堂，仅有4人一桌的位置
- 去吃饭，有1人~4人甚至更多的情况
- 如何分配座位？
- 根据增加的功能酌情加分，比如加入图形界面。



推荐参考材料

《C++面向对象程序设计（第三版）》 by 谭浩强 （主要参考教材）

《C++程序设计（第三版）》 by 谭浩强

《C++程序设计（第二版）》 by 王挺 等

《C++ Primer Plus》 by Stephan Prata, 张海龙 等译

课件所用素材来自：包括但不限于谭浩强、王挺、程磊、王秀, 王灿辉 等老师。

- C++ documentation — DevDocs (<https://devdocs.io/cpp/>)
- C++ Language Reference | Microsoft Learn
- 善用 谷歌、知乎、GitHub、Stack Overflow、B站等网站工具
- 大模型工具：DeepSeek, ChatGPT等

大模型?



大模型?



孙悟空为什么要三打镇关西?



孙悟空三打镇关西是因为他要救唐僧，而镇关西是唐僧的前世仇人。



重新回答



16:23

孙悟空为什么要三打镇关西？

孙悟空没有三打镇关西，三打镇关西是鲁智深的故事。

首先，孙悟空是《西游记》中的主角之一，他是一只猴子，拥有神奇的能力和智慧。

其次，《西游记》中三打镇关西的是鲁智深，为了救金氏父女而痛打镇关西。

最后，鲁智深和孙悟空是完全不同的两个角色，没有孙悟空三打镇关西的说法。



有问题尽管问我~



大模型？

孙悟空为什么要三打镇关西？

孙悟空并没有三打镇关西，这是《水浒传》中的情节，发生在鲁智深身上。

首先，孙悟空是《西游记》中的主角之一，他生性聪明、嫉恶如仇，但同时又心高气傲，比较冲动。

其次，《水浒传》中，鲁智深为了救金氏父女三拳打死了镇关西，而《西游记》中孙悟空则是为了帮助唐僧解围才与镇关西争吵起来。

因此，这个问题是违反事实的。

大模型?

孙悟空为什么要三打镇关西?

- ✓ 正在搜索: 孙悟空三打镇关西原因
- ✓ 正在为你生成答案...

您可能有些混淆了。在《西游记》中，孙悟空并没有三打镇关西的情节。您可能是将《水浒传》中的鲁智深拳打镇关西与孙悟空混淆了¹²。在《水浒传》中，鲁智深（鲁提辖）因为镇关西欺压百姓，强抢民女，所以鲁智深义愤填膺，决定为民除害，这才有了拳打镇关西的情节¹²。如果您有其他问题或需要更多的信息，请告诉我！

了解详细信息: [1. zhidao.baidu.com](https://zhidao.baidu.com) +4 更多

1 共 30 ●



大模型?

```
    }, speed);  
  }  
  
  speedControl.addEventListener('input', () => {  
    if (isRolling) {  
      clearInterval(interval);  
      startRoll();  
    }  
  });  
</script>  
</body>  
</html>
```

主要特点:

1. 炫酷科幻风背景: 使用深蓝渐变背景配合发光文字效果
2. 动态速度控制: 滑轮可实时调节名字滚动速度 (最右最快)
3. 视觉反馈: 点击按钮时产生缩放动画, 选中时文字放大并变为科技蓝
4. 响应式设计: 自动适配不同屏幕尺寸
5. 流畅动画: 所有状态变化均带有平滑过渡效果

使用时只需:

1. 将提供的姓名完整粘贴到 `names` 数组中
2. 保存为 `.html` 文件
3. 双击打开即可使用

随机抽奖小程序

大模型?



1. 教材必须准确定位, 合理取舍内容。 写书首先要弄清楚本书是为什么人写的, 他们学习C++的目的是什么, 应当学到什么程度。 应当声明: 本书的对象不是C++的专业开发人员, 而是高校各专业的大学生, 他们并无从事程序设计的实际经验, 将来也不一定是C++的专业开发人员。 众所周知, 研制C++的初衷是为了解决大型软件开发中遇到的问题, 提高软件的开发效率。 只有参加过研制大型软件的人才能真正体会到C++的优越性, 并善于利用C++的独特机制去进行软件开发。

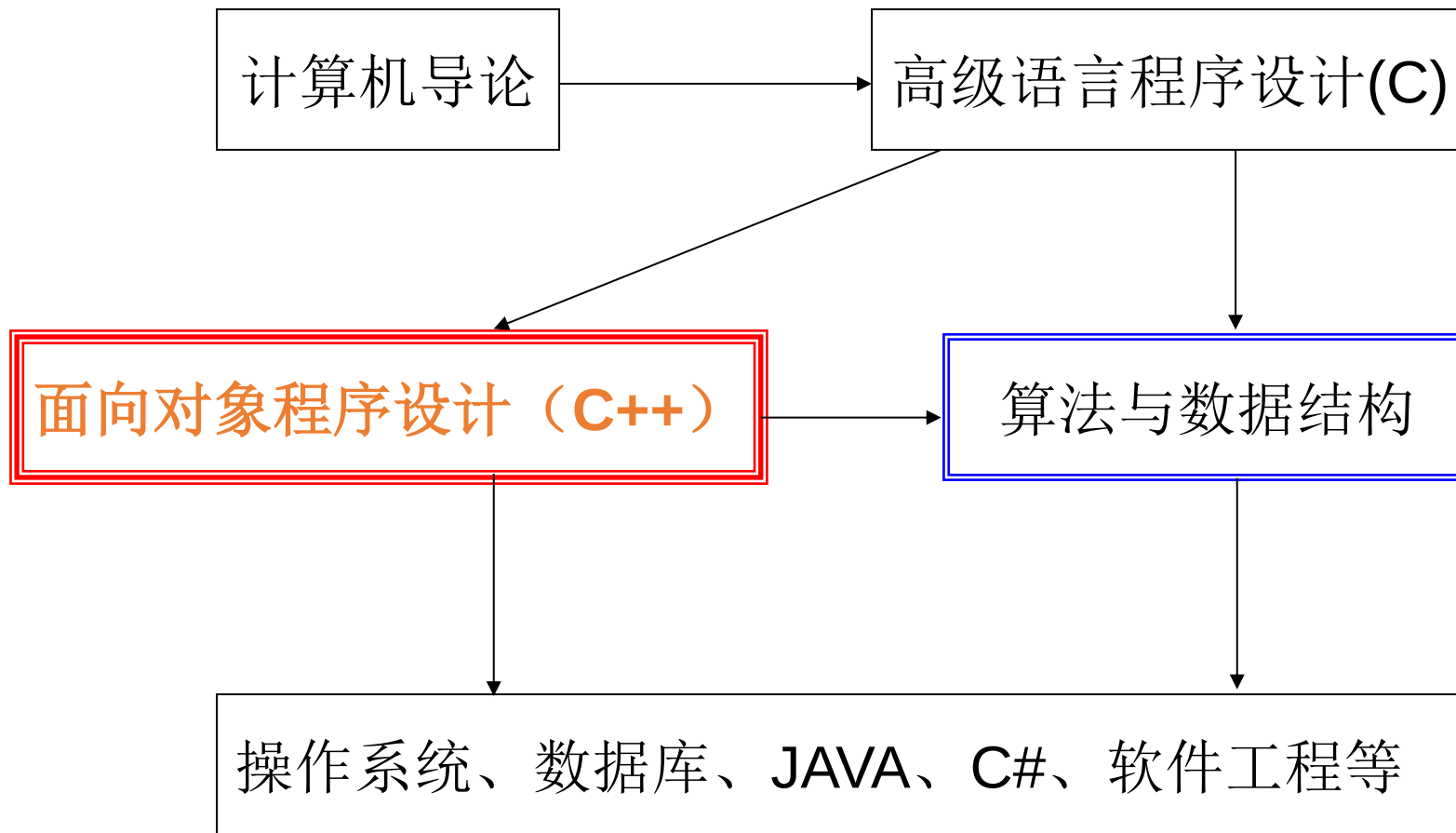
大学生在学校学习期间学习C++的目的是了解面向对象程序设计方法, 能够利用C++编写相对简单的程序, 为以后进一步学习和应用打下良好的基础。 不可能设想, 通过几十小时的学习, 能使一个没有程序设计基础的人变成一个熟练的C++开发人员。 应当有一个实事求是的分析和估计。 因此, 本书着力于使读者对C++的全貌和作用有基本的认识, 用容易理解的方法讲清楚有关的基本概念和基本方法, 而不去罗列C++语法中各种烦琐的细节。 否则将会使篇幅过大(许多国外的书厚达近千页), 而且会冲淡重点, 主次不分, 使读者感到枯燥无味。

谭浩强教材: 前言

教学内容

- 绪论
- C++ 基础知识
 - 数据类型、表达式
 - 输入与输出
 - 控制结构
 - 函数
 - 数组
 - 指针
 - 结构/联合/枚举
 - 链表
- 面向对象的程序设计
 - OOP的基本概念
 - 类与对象
 - 运算符重载
 - 继承与派生
 - 多态性与虚函数
 - 文件和流
 - 其它工具

课程定位



有最好的编程语言?

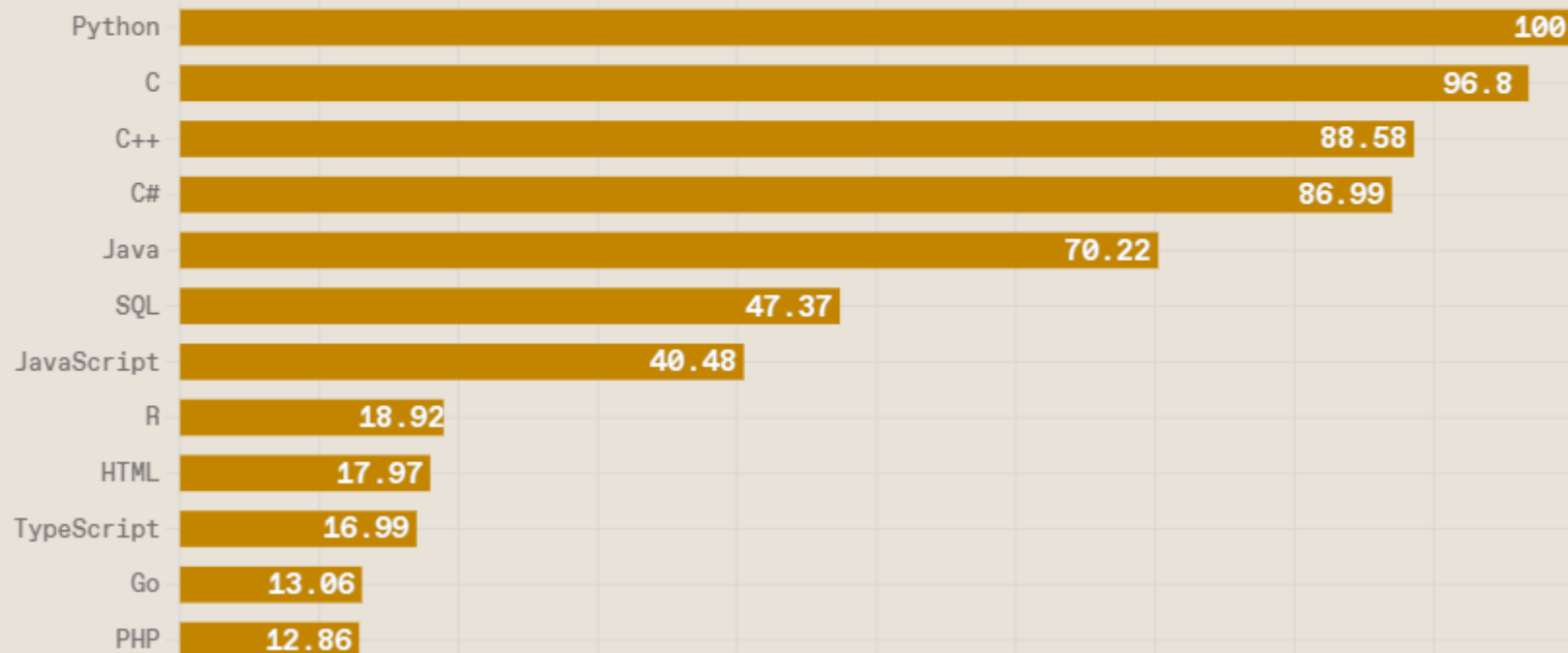
Top Programming Languages 2022

Click a button to see a differently weighted ranking

Spectrum

Jobs

Trending

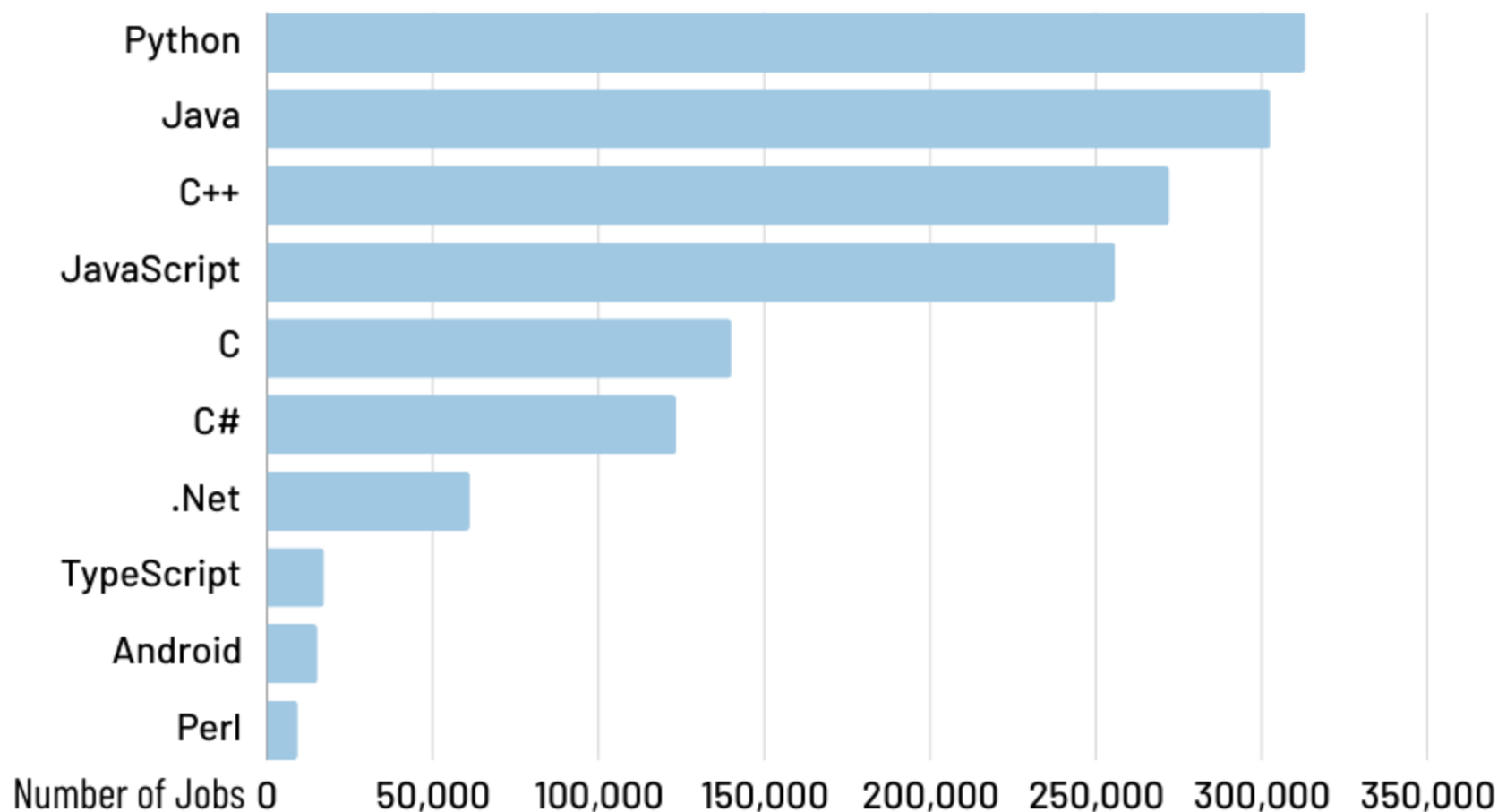


Cr: IEEE Spectrum



Most in-demand programming languages of 2024

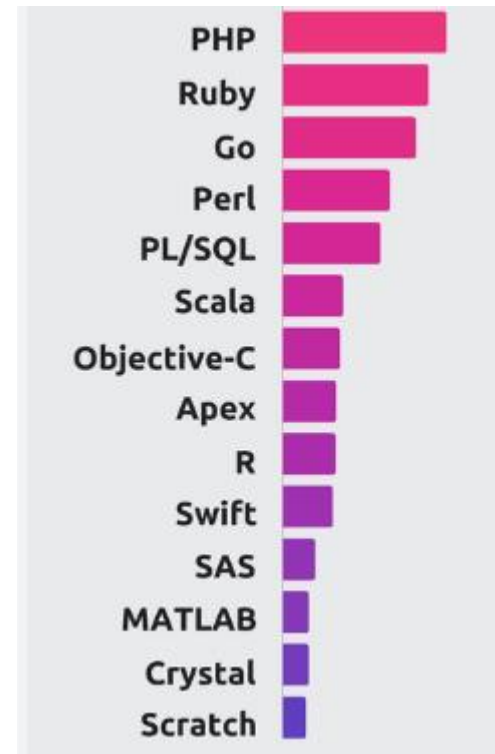
Based on LinkedIn job postings in the US



By: CodingNomads

有最好的编程语言？

- 有最好的编程语言吗？在软件领域永远没有最好，只有更好。能解决问题的都是好方法或是好语言。
- 真正学好一种计算机语言以后，再学同一类语言会非常轻松。
- 多动手实践。Talk is cheap, show me the code.





本节提纲

- 计算机系统概述
- 程序设计的基本概念与方法
- 程序设计语言及其发展
- 结构化程序设计方法与面向对象程序设计方法概述
- C语言与C++语言
- 面向对象程序设计？

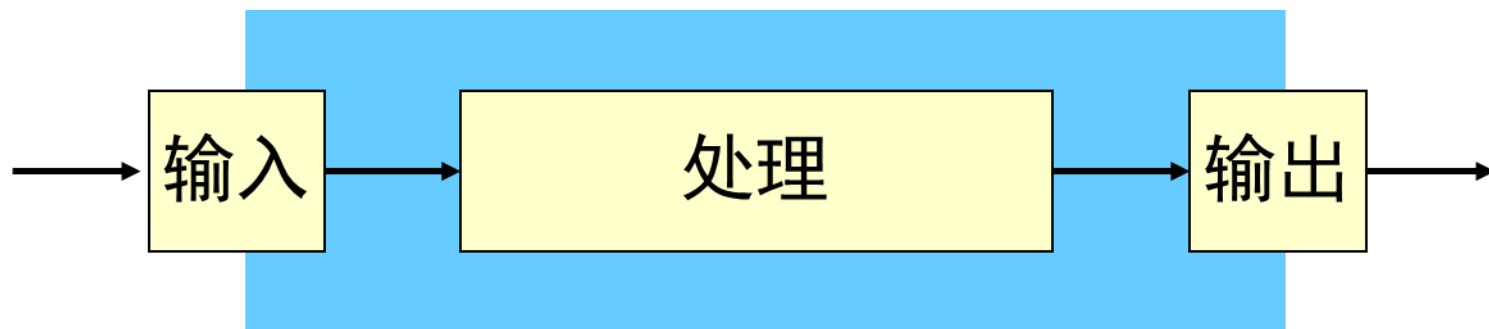


绪论——

计算机系统概述

计算机系统概述

- 什么是计算机系统?
 - 机器系统
 - 能接收和存储信息
 - 能对数据进行处理和计算
 - 能输出结果



计算机系统的组成

硬件

主板

CPU

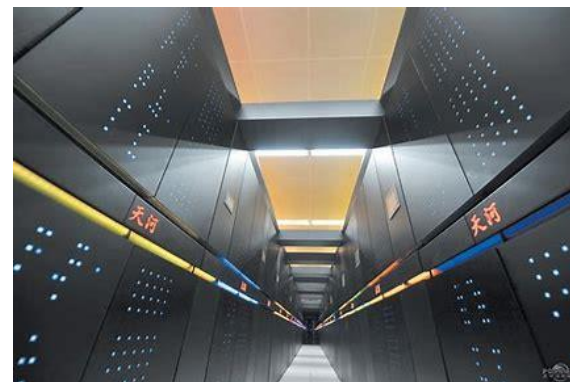
内存

硬盘

显示器

键盘

...

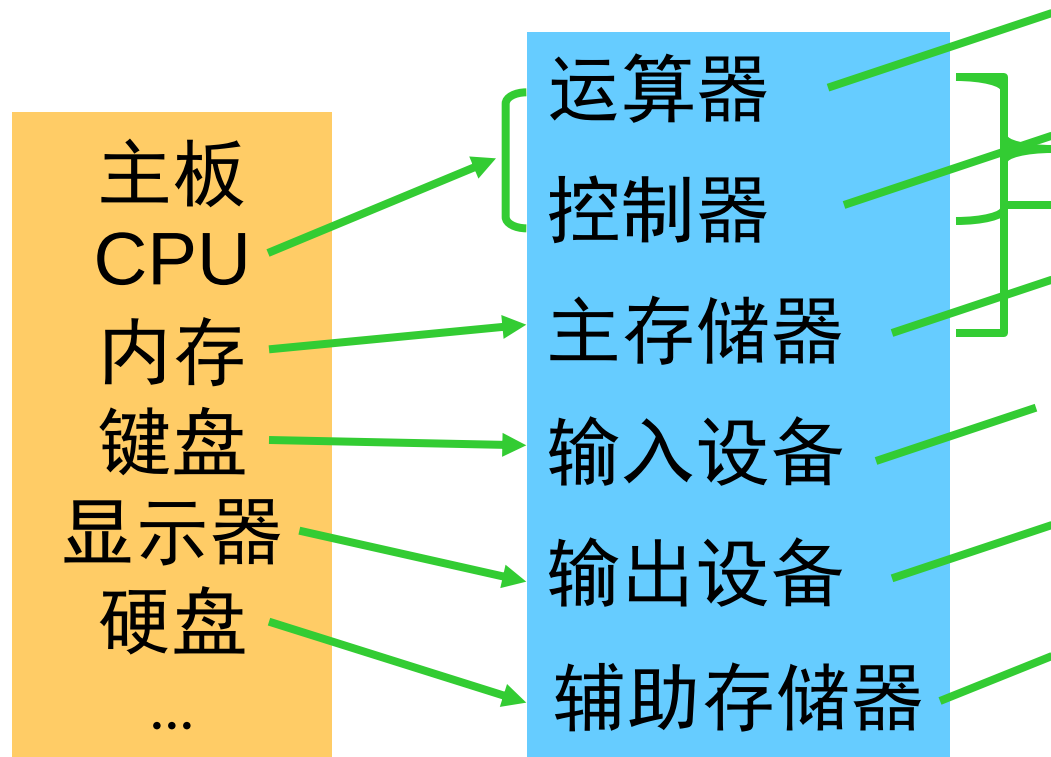


程序和文档

软件

计算机硬件

■ 是软件实现的基础



硬件

分类

按程序的要求，控
制数据流进行管术运
由由外理哭
处理机 司信息，
控制器
将计算机中二进制
信息变换为用户所
存储主存储器难以
容纳、又为程序执
行所需要的大量文
件信息



冯.诺依曼体系结构

- 二进制逻辑
- 程序存储执行
- 计算机由五部分组成

人列计算机?

计算机软件

- 计算机系统中的指挥者
- 分类



上述分类是交叉和变化的

- 一个软件可以在一个系统中是系统软件，在另一个系统中是支撑软件
- 在同一系统中，一个软件可以既是系统软件，也是支撑软件



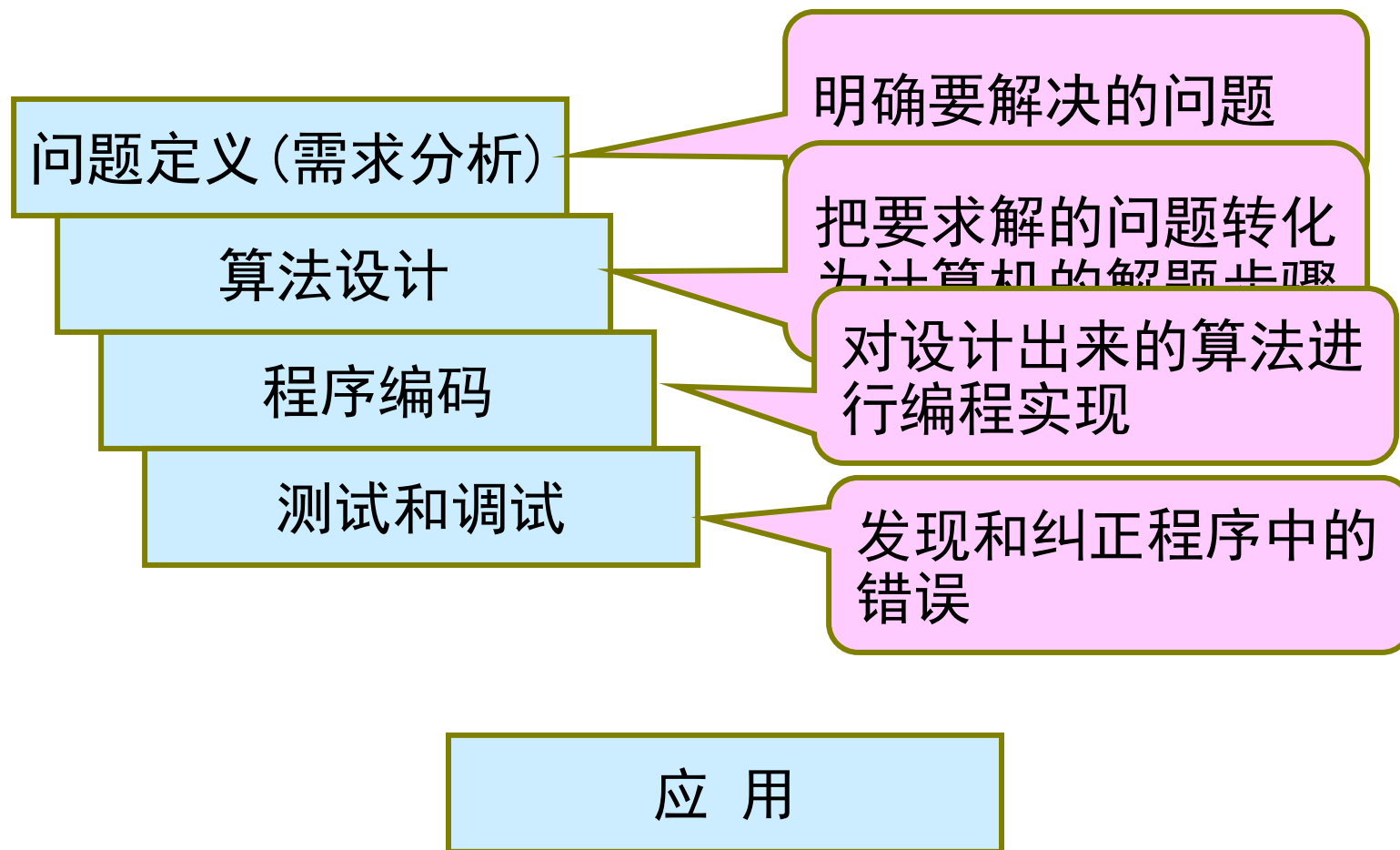
绪论——

程序设计基本概念



- **程序设计**是构造程序求解问题的方法和过程。

问题求解过程





算法的例子

有2个鸡蛋，在总共有100层的楼上往下扔，以此来测试鸡蛋的硬度。比如鸡蛋在第9层没有摔碎，在第10层摔碎了，那么鸡蛋不会摔碎的临界点就是9层。

问：如何用最少的尝试次数，测试出鸡蛋不会摔碎的临界点？

算法

- 由有限个步骤组成的求解问题的方法
- 特点
 - 有穷性：执行有穷个步骤后必须终止
 - 确定性：每个步骤都是无二义的
 - 可行性：每个步骤都可以在有限时间内完成
 - 输入：一般有输入信息
 - 输出：一般有一个或多个输出信息

算法描述

■ 描述方式

■ 文字描述

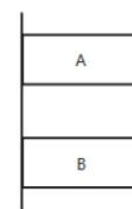
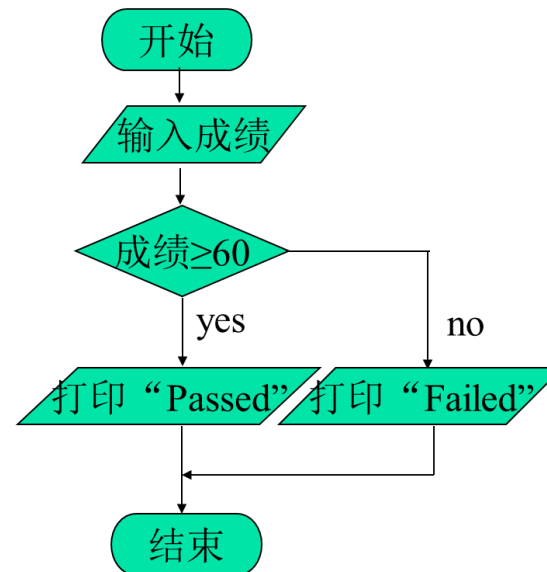
自然语言/伪代码

■ 图形描述

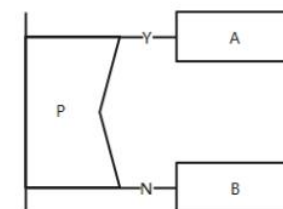
流程图、盒图，PAD图等

■ 程序语言描述

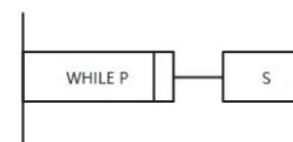
计算机程序，如C++



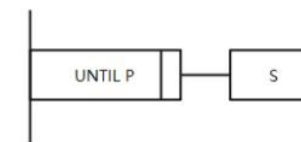
1) 顺序型



2) 选择型



3) WHILE重复型



4) UNTIL重复型

PAD

程序

- 程序是对计算任务的**处理对象**和**处理规则**的描述，它规定了计算机执行的动作和动作的顺序

- 组成

- 数据

程序处理的对象，具有**数据结构**

- 操作

处理数据的操作步骤，即**算法**

程序 = 数据结构 + 算法

Niklaus Wirth

程序设计语言

- 描述数据和对数据的操作
- 分类



表示方法更接近于待解问题的表示方法，需要编译才能运行

使用一组助记符编码，需要汇编才能运行

用机器基本指令编码，二进制形式

机器语言（用二进制代码表示）

- 是计算机能够直接识别的语言，是特定计算机的自然语言，由计算机的硬件设计定义，通常由一系列数字组成，最终简化为0和1，能让计算机执行最基本的操作。

```
10111000 00000010 00000000  
00000101 00000110 00000000  
00101101 00000011 00000000
```



已知 $a=2$, $b=6$, $c=3$,
求 $y=a+b-c$ 的值。

汇编语言（用符号表示）

- 用类似英文缩写的助记符来表示计算机的基本操作，对程序员比较清晰，但计算机不能直接理解，需由汇编器转换为机器语言。

```
MOV AX, 2  
ADD AX, 6  
SUB AX, 3
```

高级语言（类似自然语言）

- 高级语言类似日常英语，包含有常用的数学符号，一条语句完成大量任务。把高级语言转化为机器语言的翻译程序称为编译器 (compilers)。

```
a=2;  
b=6;  
c=3;  
y=a+b-c;
```

示例：已知 $a=2$ ， $b=6$ ， $c=3$ ，求 $y=a+b-c$ 的值。

```
10111000 00000010 00000000
00000101 00000110 00000000
00101101 00000011 00000000
```

```
MOV  AX, 2
ADD  AX, 6
SUB  AX, 3
```

```
a=2;
b=6;
c=3;
y=a+b-c;
```




是否有更接近自然语言的程序设计语言？

Wenyan:

吾有一數。曰三。名之曰「甲」。
為是「甲」遍。
吾有一言。曰「問天地好在。」。書之。
云云。

Equivalent JavaScript:

```
var n = 3;  
for (var i = 0; i < n; i++) {  
    console.log("問天地好在。");  
}
```

Output:

問天地好在。
問天地好在。
問天地好在。

截至 n 的斐波那契数列

>>> 套路 斐波那契数列(n):

>>> a, b = 0, 1

>>> 只要 a < n:

>>> 打印(a, 结尾=' ')

>>> a, b = b, a+b

>>> 打印()

>>> 斐波那契数列(1000)

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987

草蟒 <https://www.grasspy.cn/>

文言 wenyan-lang (Lingdong Huang from CMU)
<https://github.com/wenyan-lang/wenyan>

程序语言的发展

- 程序语言越低级，则必须对过程描写的越具体，指令也就越接近机器的硬件逻辑。
- 相反，程序语言越高级，就越接近对问题的描述和表达，因而更直观，更容易被人们理解。
- 程序语言的发展，总是从低级到高级，也就是从具体表述到抽象描述，高级语言的抽象描述最后都自动转化成机器的具体实现。



绪论——

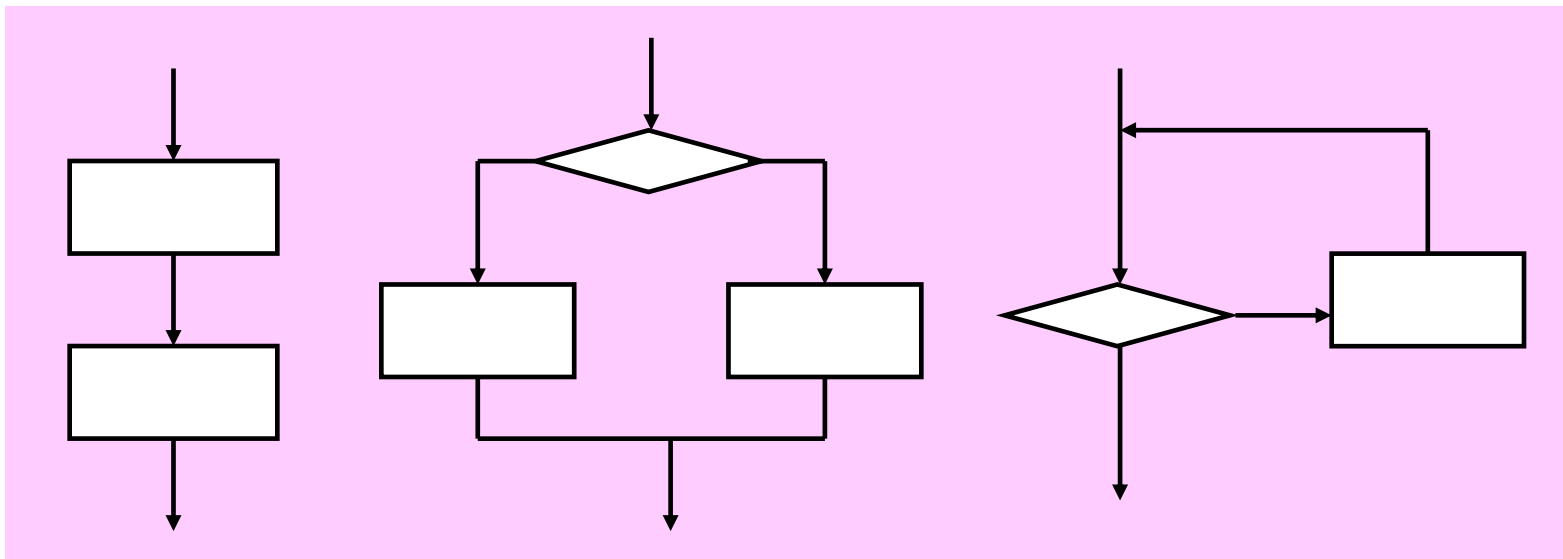
程序设计方法

面向过程的程序设计（结构化程序设计方法）

- 面向过程的程序设计思想的核心是功能分解，通常采用自顶向下的方法进行程序设计，即传统的结构化程序设计(Structured Programming, SP) 方法.

面向过程的程序设计（结构化程序设计方法）

- E.Dijkstra等人，1972年
- **结构定理**：任何程序逻辑都可以用**顺序**、**选择**和**循环**三种基本结构表示



面向过程的程序设计（结构化程序设计方法）

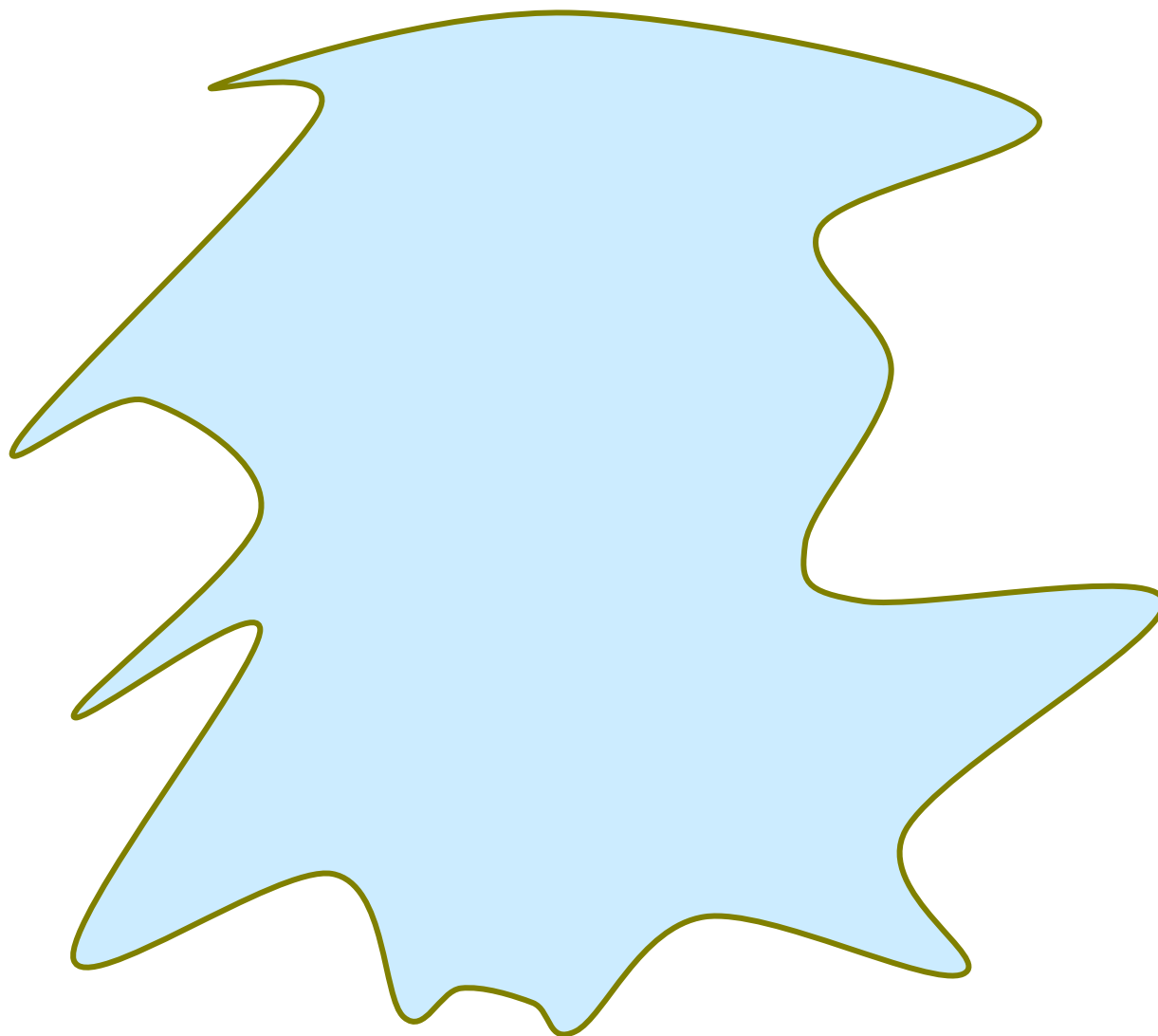
- 原则

- 自顶向下，逐步求精

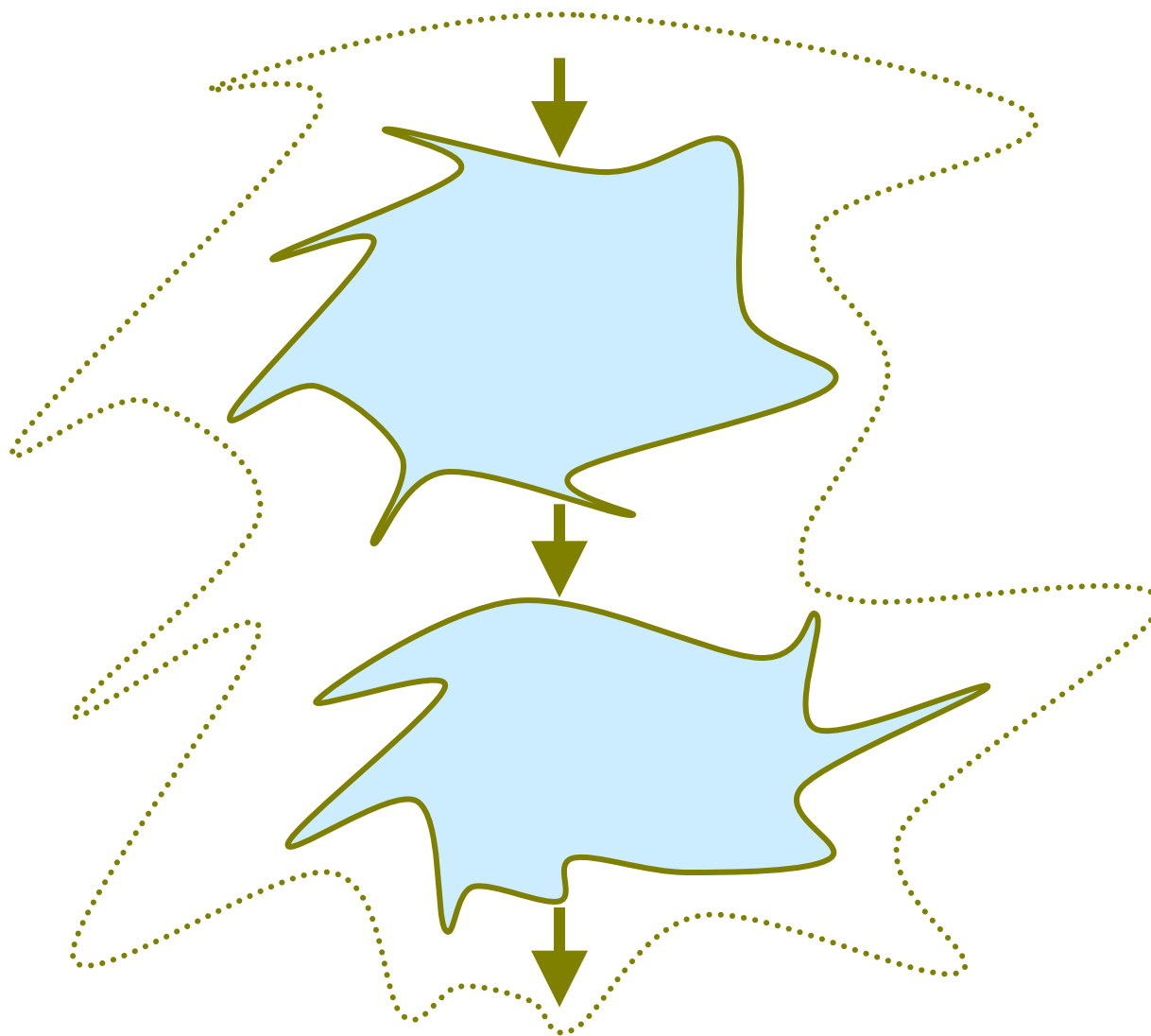
1. 1971年由Niklaus Wirth提出
子问题用抽象数据和抽象操作来描述
2. 如此反复，直至获得计算机能理解的程序为止



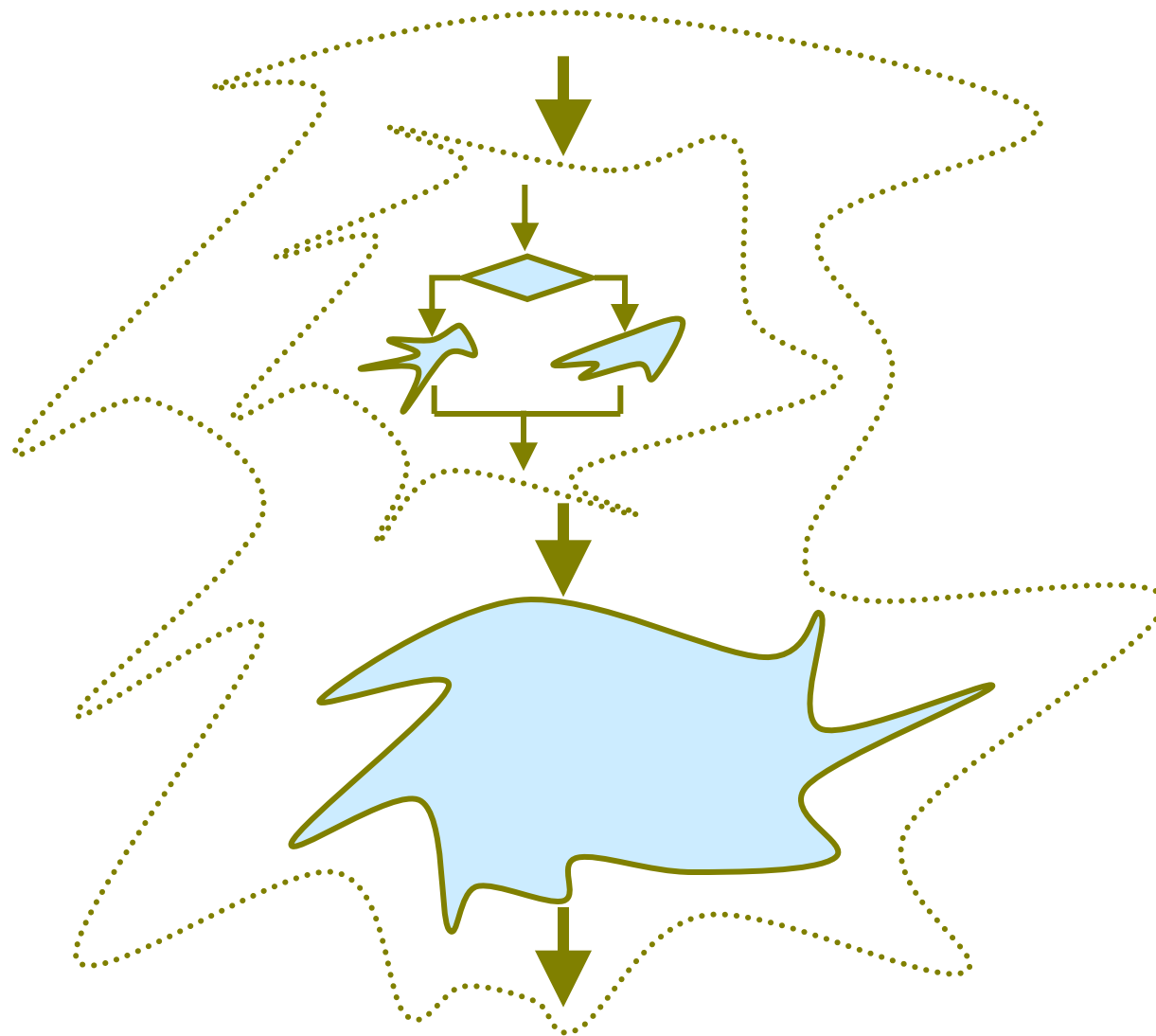
自顶向下，逐步求精



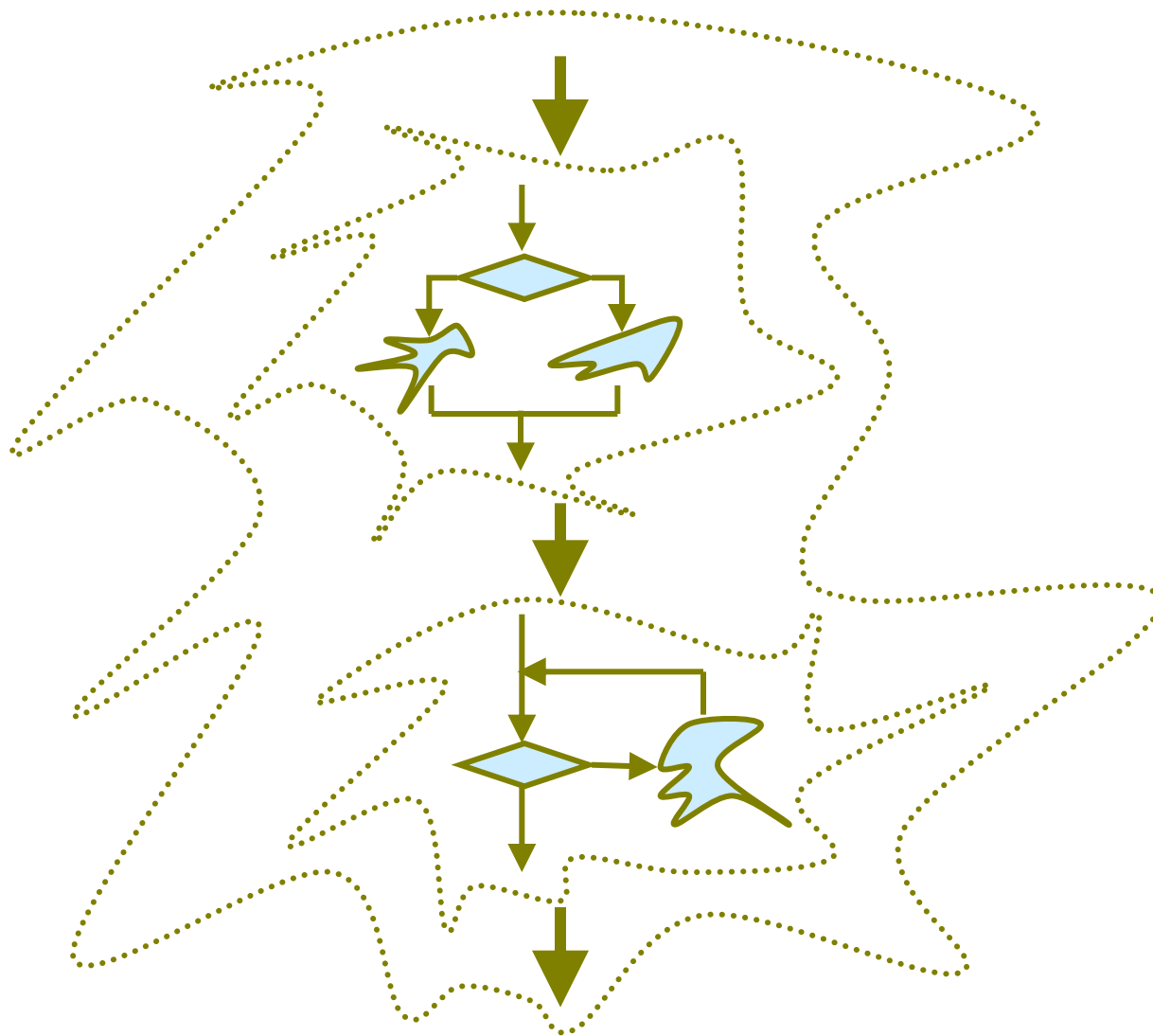
自顶向下，逐步求精



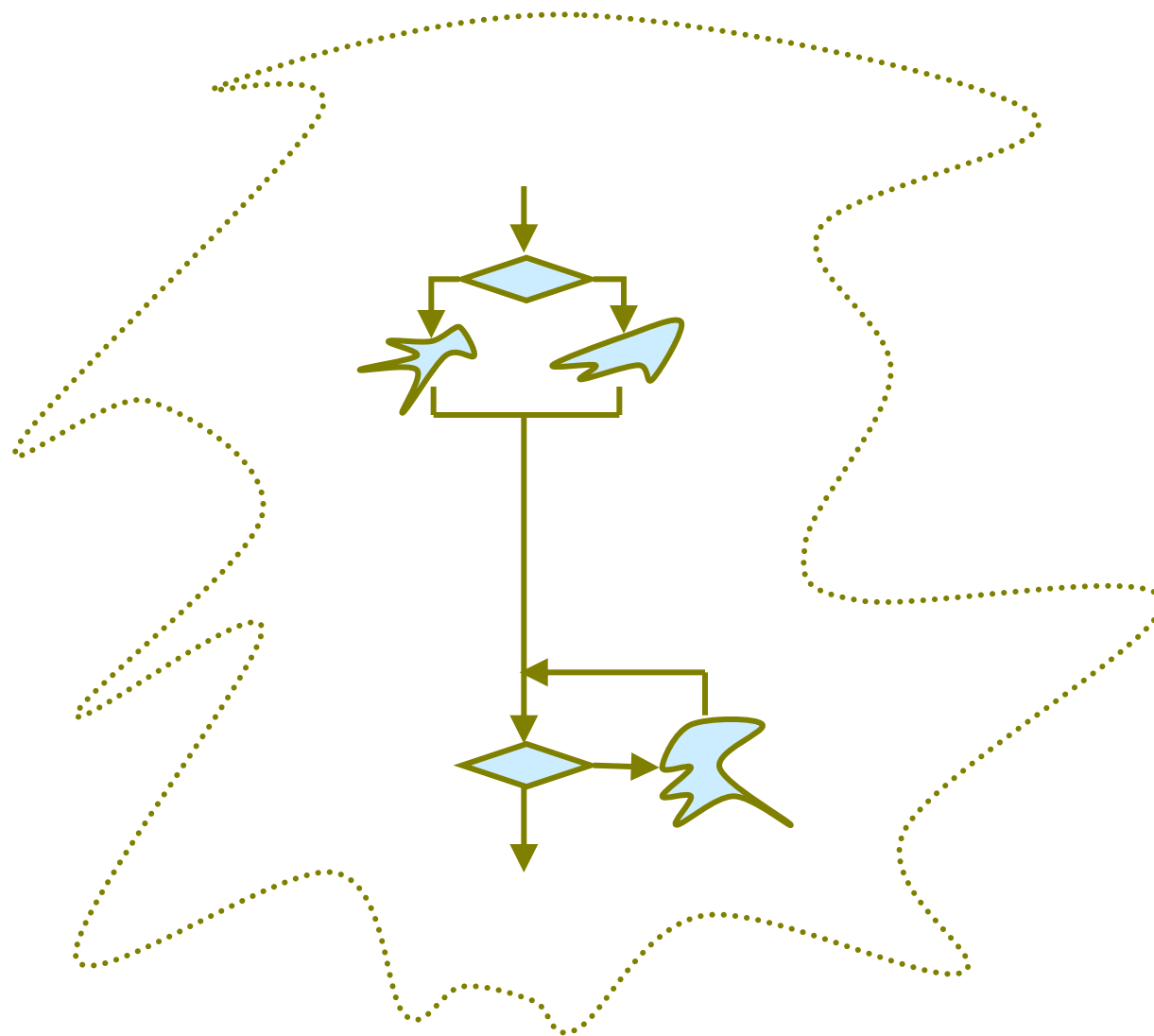
自顶向下，逐步求精



自顶向下，逐步求精



自顶向下，逐步求精



面向过程的程序设计（结构化程序设计方法）

- 设计思路
 - 自顶向下、功能分解、逐步求精。采用模块分解与功能抽象，自顶向下、分而治之。
- 程序结构：
 - 按功能划分为若干个基本模块。
 - 各模块间的关系尽可能简单，功能上相对独立；每一模块内部均是由顺序、选择和循环三种基本结构组成。
 - 其模块化实现的具体方法是使用子程序。

面向过程的程序设计（结构化程序设计方法）

- 典型语言

- Pascal, Niklaus Wirth教授, 1971年
- C, Dennis Ritchie和Ken Thompson, 1972年左右



面向过程的程序设计（结构化程序设计方法）

优点：

有效地将一个较复杂的程序系统设计任务分解成许多易于控制和处理的子任务，便于开发和维护。

面向过程的程序设计（结构化程序设计方法）

- **缺点：** 可重用性差、数据安全性差、难以开发大型软件

- **原因分析：**

- ① 把数据和处理数据的过程分离为相互独立的实体。
- ② 当数据结构改变时，所有相关的处理过程都要进行相应的修改。
- ③ 每一种相对于老问题的新方法都要带来额外的开销。
- ④ 对于大型的软件，很难用过程来描述和实现，模块之间调用关系复杂且互相依赖度强，开发和维护也都很困难。

因此，我们需要面向对象的程序设计

面向对象程序设计方法

■ 由来

心理学研究表明，把客观世界看作由许多对象组成，对象具有其属性和行为，对象之间存在着各种联系，这样能够更好地刻画问题域，也更接近人类的自然思维方式

面向对象的程序设计

- 面向对象程序设计

(Object-Oriented Programming, OOP) 方法强调以问题域（现实世界）中的事物为中心来思考和认识问题，并按照事物的本质特征将其抽象为对象，以作为构成软件系统的基础。

面向对象程序设计方法的特点：

- (1) OOP方法以 “对象” 或 “数据” 为中心。
- (2) 引入了 “类” 的概念。
- (3) OOP方法的模块性与继承性保证了新的应用程序设计可在原有对象的数据类型和功能的基础上通过重用、扩展和细化来进行，而不必从头做起或复制原有代码。

面向对象程序设计方法

- 结构化程序的基本单位是函数，面向对象程序的基本单位是对象
- 对象是对数据和函数的封装
- 面向问题的思考方式，与现实世界的对应更直接
- 易于扩展、可维护性好

面向对象程序设计方法

- 将数据及对数据的操作方法封装在一起，作为一个相互依存、不可分离的整体——对象。
- 对同类型对象抽象出其共性，形成类。
- 类通过一个简单的外部接口，与外界发生交互。
- 对象与对象之间通过消息进行通讯。

面向对象程序设计方法

优点：

“面向对象技术是70年代以来最激动人心的革新之一” [Maurice Wilkes]

- 程序模块间的关系更为简单，程序模块的独立性、数据的安全性就有了良好的保障。
- 通过继承与多态性，可以大大提高程序的可重用性，使得软件的开发和维护都更为方便。

面向对象程序设计方法

- 步骤
 1. 找出问题中的对象和类
 2. 确定每个对象和类应具备的属性和方法
 3. 确定对象与类之间的对应关系、对象之间相互作用的方式以及类之间的关系
 4. 用程序代码实现这些对象和类

面向对象程序设计方法

- 典型语言

- **Simula**, Kisten Nygaard和Ole-Johan Dahl, 1966年
- **Smalltalk**, Alan Kay等, 20世纪70年代
- **C++**, Bjarne Stroustrup, 20世纪80年代初
- **Delphi**, Borland公司, 1995年
- **Java**, Sun公司, 1995年

On the basis of	Procedural Programming	Object-oriented programming
Definition	It is a programming language that is derived from structure programming and based upon the concept of calling procedures. It follows a step-by-step approach in order to break down a task into a set of variables and routines via a sequence of instructions.	Object-oriented programming is a computer programming design philosophy or methodology that organizes/ models software design around data or objects rather than functions and logic.
Security	It is less secure than OOPs.	Data hiding is possible in object-oriented programming due to abstraction. So, it is more secure than procedural programming.
Approach	It follows a top-down approach.	It follows a bottom-up approach.
Data movement	In procedural programming, data moves freely within the system from one function to another.	In OOP, objects can move and communicate with each other via member functions.
Orientation	It is structure/procedure-oriented.	It is object-oriented.
Access modifiers	There are no access modifiers in procedural programming.	The access modifiers in OOP are named as private, public, and protected.
Inheritance	Procedural programming does not have the concept of inheritance.	There is a feature of inheritance in object-oriented programming.
Code reusability	There is no code reusability present in procedural programming.	It offers code reusability by using the feature of inheritance.
Overloading	Overloading is not possible in procedural programming.	In OOP, there is a concept of function overloading and operator overloading.
Importance	It gives importance to functions over data.	It gives importance to data over functions.
Virtual class	In procedural programming, there are no virtual classes.	In OOP, there is an appearance of virtual classes in inheritance.
Complex problems	It is not appropriate for complex problems.	It is appropriate for complex problems.
Data hiding	There is not any proper way for data hiding.	There is a possibility of data hiding.
Program division	In Procedural programming, a program is divided into small programs that are referred to as functions.	In OOP, a program is divided into small parts that are referred to as objects.
Examples	Examples of Procedural programming include C, Fortran, Pascal, and VB.	The examples of object-oriented programming are - .NET, C#, Python, Java, VB.NET, and C++.



福州大学
FUZHOU UNIVERSITY



绪论——

C 与 C++

C语言的历史

- 起源于Algol 60
- 在Algol 60的基础上发展出CPL语言
- 1967年发展为BCPL，用于编写操作系统和编译器
- 1970年贝尔实验室将BCPL改良成B，并用B语言实现了第一个UNIX操作系统
- 1972年左右，Dennis Ritchie和Ken Thompson在贝尔实验室设计UNIX系统时，开发出了C语言

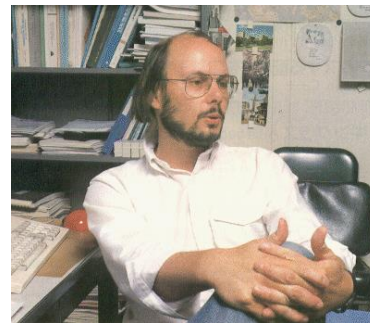


Dennis Ritchie &
Ken Thompson

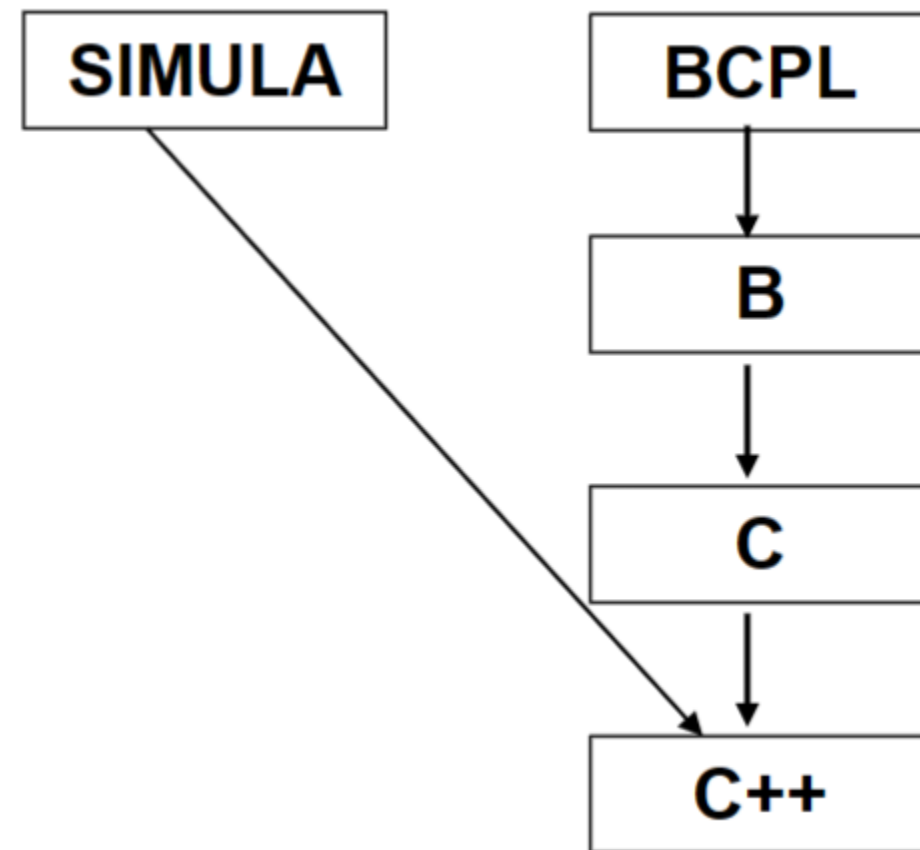
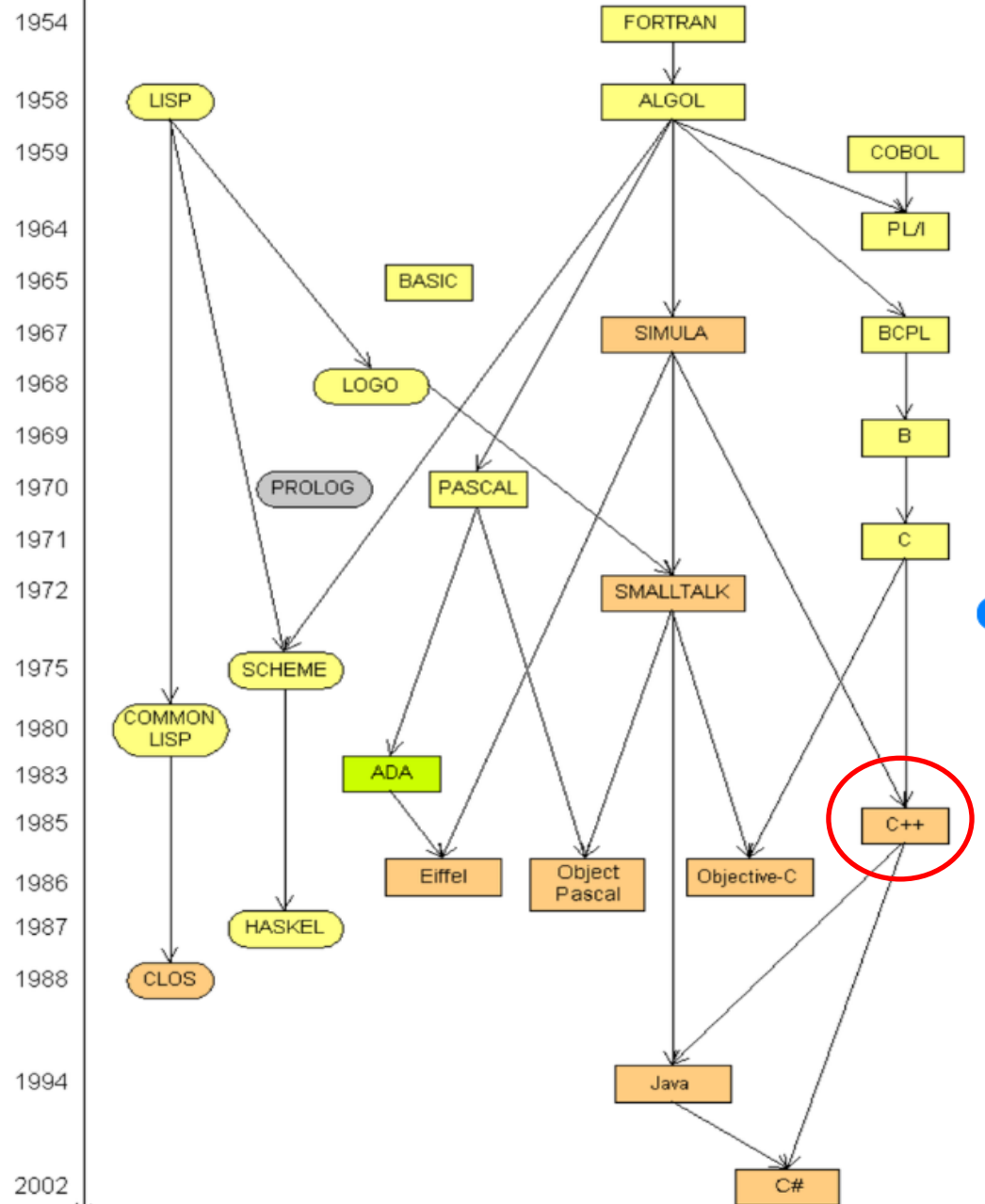
C语言的特点

- 数据类型丰富
- 运算符丰富，共34个运算符
- 支持结构化程序设计
- 方便、灵活，结合了高级语言的基本结构、语句和低级语言的实用性
- 程序生成代码质量高、程序执行效率高
- 可移植性好，可用于UNIX、WINDOWS等操作系统

从C到C++



- C++语言是从C语言发展演变而来的
- C语言毕竟是一个面向过程的编程语言，因此与其它面向过程的编程语言一样，已经不能满足运用面向对象方法开发软件的需要。
- C++正是为了解决上述问题而设计的
- C++是美国贝尔实验室的Bjarne stroustrup博士在C语言的基础上，增加了面向对象的特征，开发出来的一种面向过程与面向对象结合的程序设计语言。最初称为“带类的C”，1983年正式取名为C++



Selection of computer programming languages for developing distributed systems, Shadman Salih, 2014



Musk: 社保系统150~360岁居民信息? 可能与COBOL语言有关。

马斯克“查账”美国社保 查出“360岁老人”

光明网 2025-02-18 17:29

当地时间2月17日，美国“政府效率部（DOGE）”负责人埃隆·马斯克在社交媒体平台上表示，美国的社保系统可能存在着“有史以来规模最大的欺诈行为”。

马斯克17日在社交媒体平台公布了美国社保数据库的数据统计，显示存在着人员总数错误以及年龄数据过于离谱的问题。



Elon Musk  
@elonmusk

Follow

According to the Social Security database, these are the numbers of people in each age bucket with the death field set to FALSE!

Maybe Twilight is real and there are a lot of vampires collecting Social Security 🤔🤔

Fact Check

A 2024 report from the Social Security Administration (SSA) Inspector General confirmed that some payments were mistakenly made to dead people. But these accounted for less than 1 per cent of the \$8.6 trillion paid in benefits between 2015 and 2022.

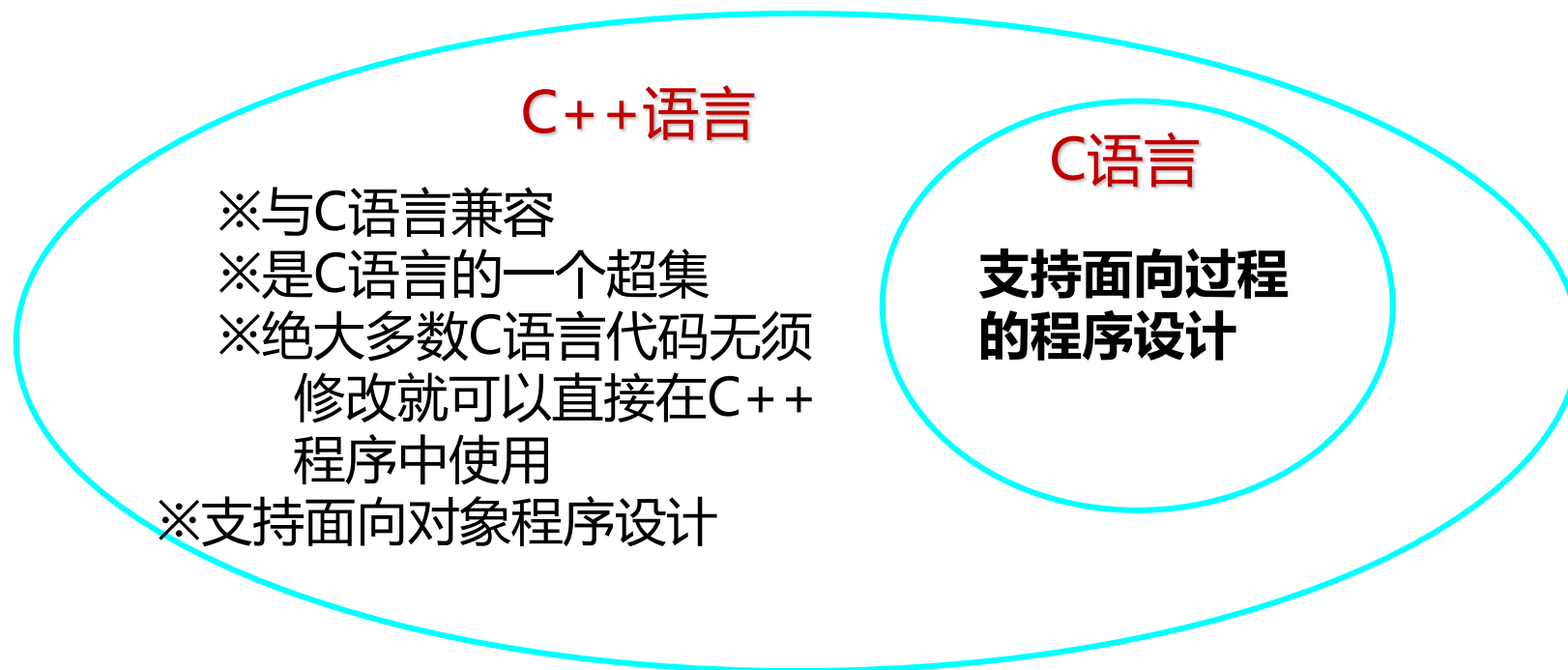
One big reason for the confusion is the SSA's outdated COBOL-based system, which doesn't handle dates properly. Missing or incomplete birthdates get a default reference point, sometimes over 150 years old. A 2023 SSA report also confirmed that "almost none" of these mistakenly listed individuals were actually receiving payments.

The SSA also has strict rules to prevent fraud. Since September 2015, it has automatically stopped payments to anyone listed as 115 years or older.

C++语言

- 在C语言中引入了面向对象的思想
- 是一种混合型语言，同时支持结构化程序设计和面向对象程序设计方法
- 兼容C语言，可不加修改地使用C语言代码
- 允许数据抽象，支持封装、继承和多态等特征
- 在原来基于过程的机制基础上,对C语言的功能做了不少扩充。

C++与C语言的关系



C++语言与C语言的本质区别： C++语言支持**面向对象**的程序设计
C语言仅支持**面向过程**的程序设计

学好C++语言的关键是掌握好面向对象的程序设计思想



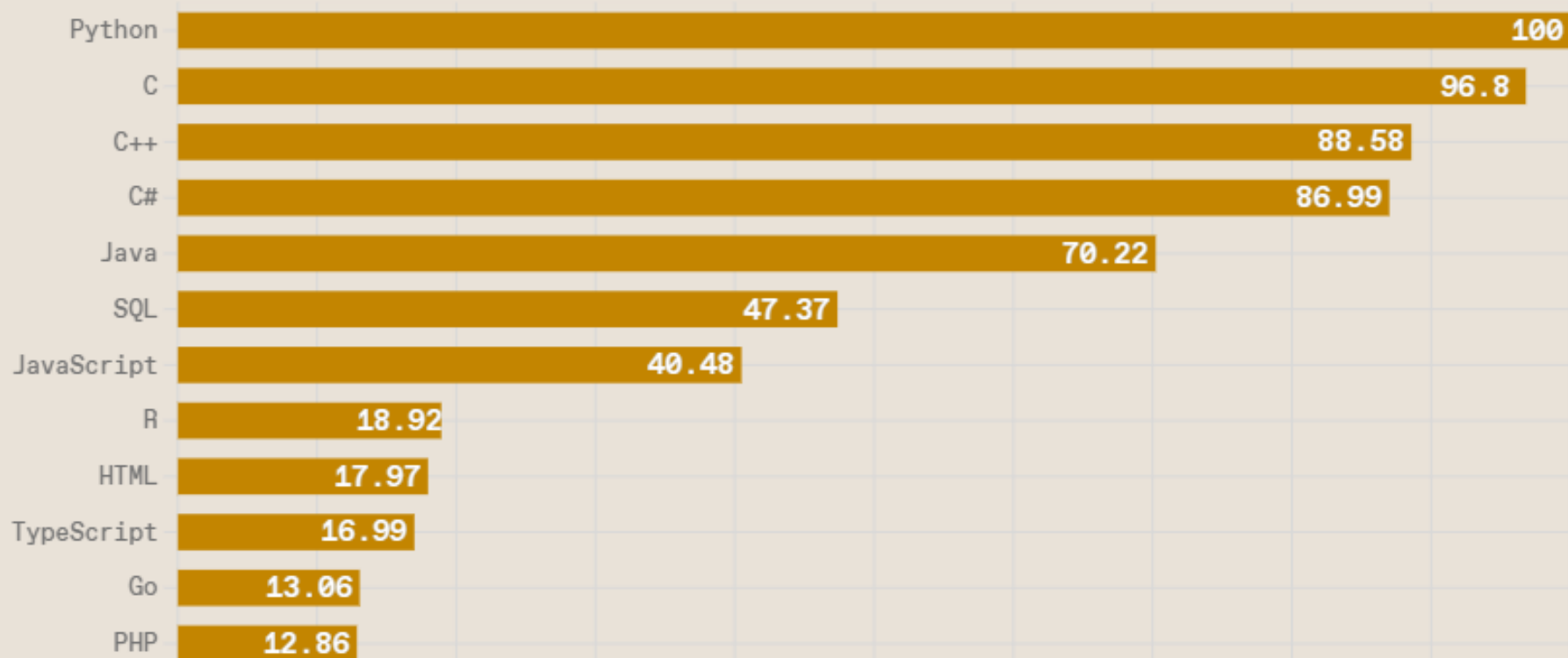
Top Programming Languages 2022

Click a button to see a differently weighted ranking

Spectrum

Jobs

Trending



Cr: IEEE Spectrum

有很多著名的软件都是基于C++语言开发的:

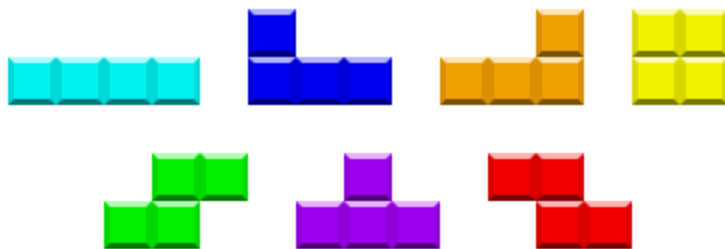
- 操作系统: 主流的3种操作系统Windows/Unix/Linux的内核都是用C语言和汇编语言写的, 很多高级特性都是用C++写的;
- 驱动程序: 涉及到调用硬件的程序几乎都会用到C/C++;
- 苹果产品: 苹果Apple的系列产品, 使用的Object-C, 一种类C++语言;
- 数据库引擎: MySQL、Oracle、SQLServer等基本上使用C、C++及汇编语言编写的;

- 图形图像软件：Adobe平台的各种软件如Photoshop、Acrobat 等；
- 浏览器软件：IE浏览器、火狐浏览器、谷歌Chrome浏览器等都是用C++编写的；
- 网络游戏：PC平台的大部分网络游戏，如《魔兽世界》、《英雄联盟》、《跑跑卡丁车》等2D、3D类游戏，均得益于C++的高性能运算和速度；
- 即时通信软件：PC平台上的QQ、阿里旺旺等；
- 通信行业后台管理系统：大型的后台，很多都使用C++；
- 应用软件：Microsoft Office、Microsoft Visual Studio等等都是用C++编写的。

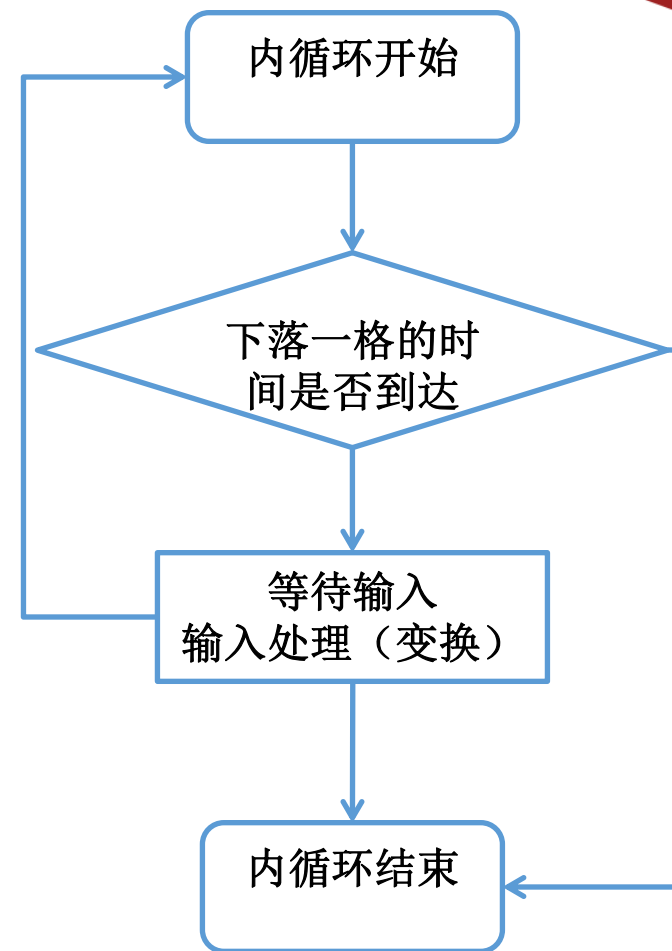
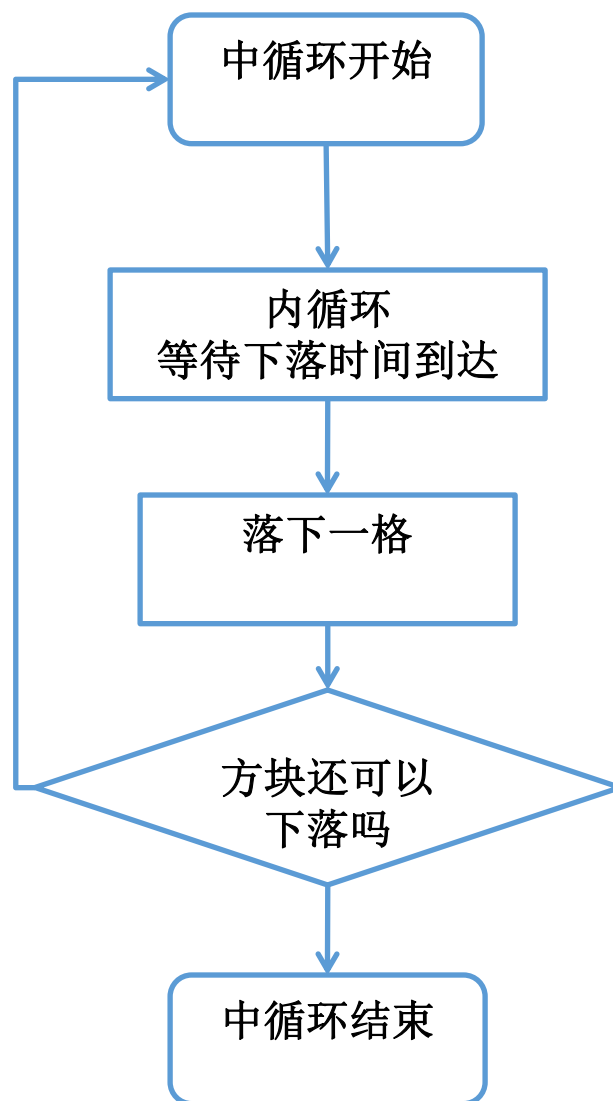
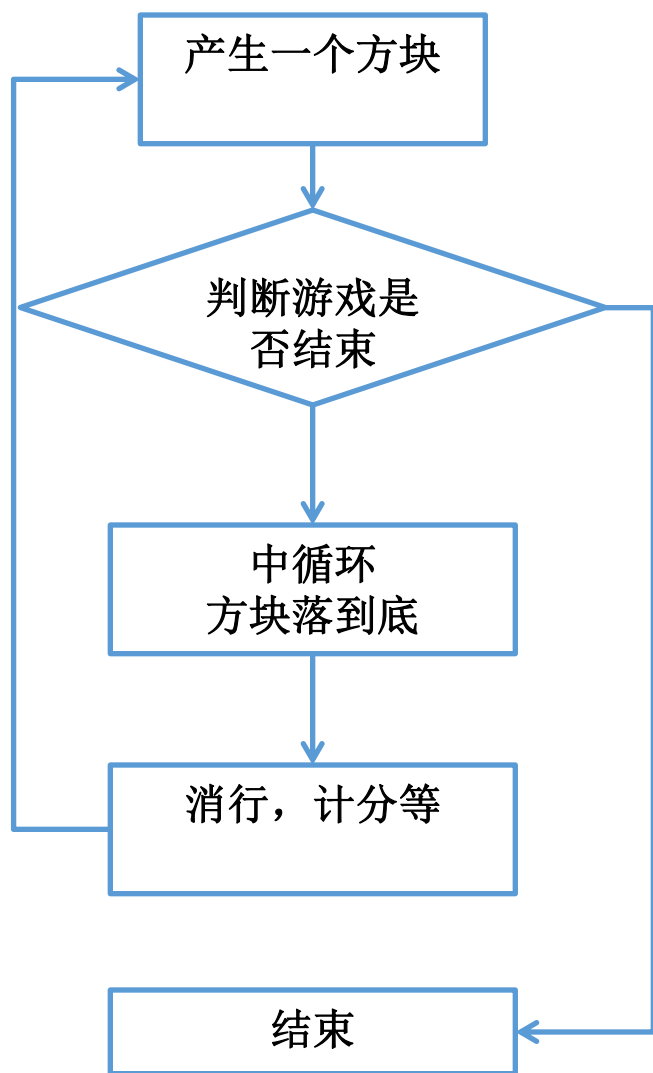
面向过程设计与面向对象设计的区别？（以俄罗斯方块为例）

俄罗斯方块的基本规则

- 1、一个用于摆放小型正方形的平面虚拟场地（容器）
- 2、一组由4个小型正方形组成的规则图形，称为方块，共有7种
- 3、随机产生单个方块到场地顶部，方块可以移动、旋转、下落并填充到场地中。每次摆放如果将场地的一行或多行完全填满，则组成这些行的所有小正方形将被消除，并且以此来换取一定的积分或者其他形式的奖励。而未被消除的方块会一直累积，并对后来的方块摆放造成各种影响。
- 4、如果未被消除的方块堆放的高度超过场地所规定的最大高度则游戏结束。



算法实现主要流程



面向过程程序设计的实现步骤

- (1) 定义全局变量，如网格数组（容器）、关数、得分等。
- (2) 定义方块的结构体，包含方块形状、位置、颜色等特性。
- (3) 定义并实现各功能模块子函数，如生成方块、旋转方块、判断方块是否到容器顶部或边缘、消行等。
- (4) 根据算法实现主函数（Main），主函数中完成对子函数的调用。

面向对象程序设计的实现步骤

- (1) 根据需求，合理地定义类。例如，在本例中，可定义出方块类与容器类。
- (2) 定义类的成员变量与成员函数。例如，方块类具有形状、大小、位置、颜色等成员变量，具有移动、旋转等成员函数。容器类具有大小、关数、分数、速度等**成员变量**，具有生成方块、判断方块是否到容器顶部或边缘、消行等**成员函数**。
- (3) 根据算法实现主函数（Main），在主函数中可对类进行实例化，并调用相关成员函数。

从C到C++

- 不要把面向对象和基于过程对立起来, 面向对象和基于过程不是矛盾的, 而是各有用途、互为补充的。在面向对象程序设计中仍然要用到结构化程序设计的知识。例如, 在类中定义一个函数就需要用结构化程序设计方法来实现。任何程序设计都需要编写操作代码, 具体操作的过程就是基于过程的。对于简单的问题, 直接用基于过程方法就可以轻而易举地解决。

C++编程简介

- C++集成开发环境 (IDE)

- VS CODE, Microsoft

- 微软推出的轻量级代码编译器, 支持几乎所有主流的开发语言的语法高亮、智能代码补全、自定义热键、括号匹配、代码片段、代码对比 Diff、GIT 等特性, 支持插件扩展, 并针对网页开发和云端应用开发做了优化。(免费, 开源)

- C++ Builder, Borland

- Visual Age For C++, IBM

- Visual C++, Microsoft



运行和调试

JavaScript



ch1_1.cpp X

设置



变量

监视

alist: 不可用



调用堆栈

正在运行

问题 输出 调试控制台 终端

```
'--stdin=Microsoft-MIEngine-In-wdwdqafw.rrh' '--stdout=Microsoft-MIEngine-Out-himluys.0vg' '--stderr=Microsoft-MIEngine-Out-l0aoy14g.i5d' '--pid=Microsoft-MIEngine-Pid-15p5dq44.5f2' '--dbgExe=d:\mingw64\bin\gdb.exe' '--interpreter=mi'
Please input your name:
PS E:\cpp> & 'c:\Users\Lancer\.vscode\extensions\ms-vscode.cpptools-1.14.5-win32-x64\debugAdapters\bin\WindowsDebugAdapter.exe' '--stdin=Microsoft-MIEngine-In-v2drxjxz.z2z' '--stdout=Microsoft-MIEngine-Out-4czaxpre.y1g' '--stderr=Microsoft-MIEngine-Out-bvdc2dpj.vis' '--pid=Microsoft-MIEngine-Pid-5y0t11ca.nus' '--dbgExe=d:\mingw64\bin\gdb.exe' '--interpreter=mi'
Please input your name: ZhangSan
```



断点

开发C++程序的步骤

一个程序从编写到最后得到运行结果要经历以下一些步骤。

1. 用C++语言编写程序

所谓程序,就是一组计算机系统能识别和执行的指令。每一条指令使计算机执行特定的操作。用高级语言编写的程序属于“源程序”(source program)。C++的源程序是以.cpp作为后缀的(cpp是c plus plus 的缩写)。



开发C++程序的步骤

2. 对源程序进行编译

从根本上说,计算机只能识别和执行由0和1组成的二进制的指令,而不能识别和执行用高级语言写的指令。为了使计算机能执行高级语言源程序,必须先用一种称为“编译器(compiler)”的软件(也称编译程序或编译系统),把源程序翻译成二进制形式的“目标程序(object program)”。

GCC, MinGW, LLVM

开发C++程序的步骤

- 编译是以源程序文件为单位分别编译的,每一个程序单位组成一个源程序文件,如果有多个程序单位,系统就分别把它们编译成多个目标程序(在Windows系统中,目标程序以.obj(object 的缩写)作为后缀,在UNIX系统中,以.o作为后缀)。
- 编译的作用是对源程序进行词法检查和语法检查。
- 词法检查是检查源程序中的单词拼写是否有错,例如把main错拼为mian。
- 语法检查是根据源程序的上下文来检查程序的语法是否有错,例如在cout语句中输出变量a的值,但是在前面并没有定义变量a。



开发C++程序的步骤

- 编译时对文件中的全部内容进行检查,编译结束后显示出所有的编译出错信息。一般编译系统给出的出错信息分为两种,一种是错误(**error**);一种是警告(**warning**),指一些不影响运行的轻微的错误(如定义了一个变量,却一直没有使用过它)。凡是检查出错误的程序,就不会生成目标程序,必须改正后重新编译。



开发C++程序的步骤

- 3. 将目标文件连接

在改正所有的错误并全部通过编译后,得到一个或多个目标文件。此时要用系统提供的“连接程序(linker)”将一个程序的所有目标程序和系统的库文件以及系统提供的其他信息连接起来,最终形成一个可执行的二进制文件,在Windows系统中,其后缀是.exe,是可以直接执行的。



开发C++程序的步骤

- 4. 运行程序

运行最终形成的可执行的二进制文件(.exe文件),得到运行结果。

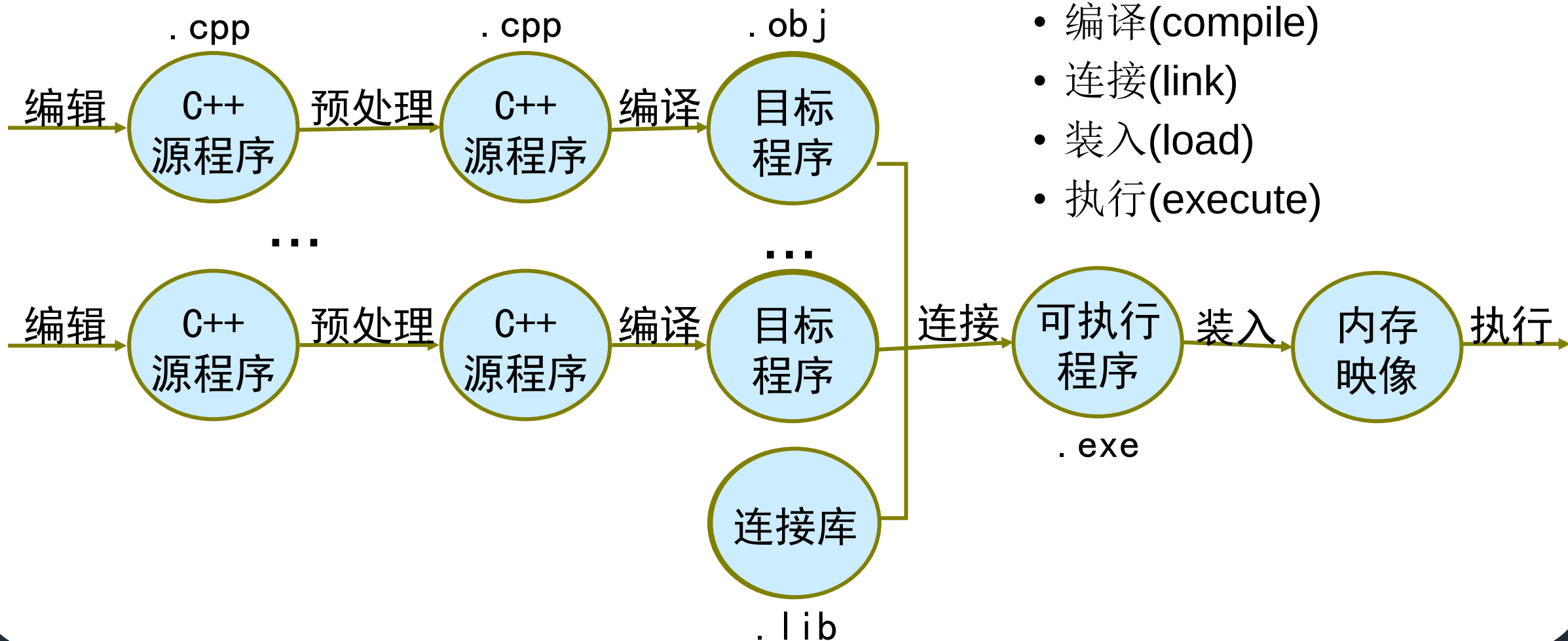
- 5. 分析运行结果

如果运行结果不正确,应检查程序或算法是否有问题。

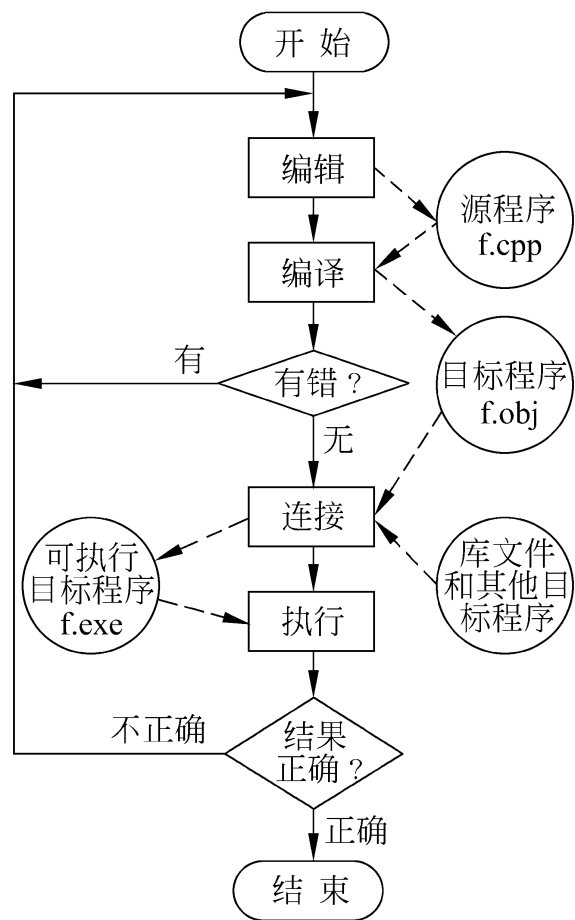
开发C++程序的步骤

- 开发C++程序的步骤

- 编辑(edit)
- 预处理(preprocess)
- 编译(compile)
- 连接(link)
- 装入(load)
- 执行(execute)



开发C++程序的步骤



- 以上过程如图所示。其中，实线表示操作流程,虚线表示文件的输入输出。例如,编辑后得到一个源程序文件f.cpp,然后在进行编译时再将源程序文件f.cpp输入,经过编译得到目标程序文件f.obj,再将目标程序文件f.obj输入内存,与系统提供的库文件等连接,得到可执行文件f.exe,最后把f.exe调入内存并使之运行。

一个简单的程序：

一个简单的C++程序：输入一个人的姓名，然后在屏幕上输出 “HELLO, * * * ! ”

```
// ch1_1.cpp: 输入姓名，然后在屏幕输出问候信息
#include <iostream>
//using namespace std;
int main( )
{
    char name[20];
    // 输出提示信息
    std::cout << "Please input your name: ";
    std::cin >> name;        // 输入姓名
    // 输出信息，并使用\n换行
    std::cout << "HELLO, " << name << "! \n";
    return 0;                // 指示程序运行
}
```

一个简单的程序：

ch1_1.cpp × 设置

ch1_1.cpp > ...

```
1  // ch1_1.cpp: 输入姓名, 然后在屏幕输出问候信息
2  #include <iostream>
3  int main( )
4  {
5      // using namespace std;
6      char name[20];
7      // 输出提示信息
8      std::cout << "Please input your name: ";
9      std::cin >> name;      // 输入姓名
10     // 输出信息, 并使用\n换行
11     std::cout << "HELLO, " << name << "! \n";
12     return 0;              // 指示程序运行
13 }
14
```

问题 输出 调试控制台 终端

```
PS E:\cpp> & 'c:\Users\Lancer\.vscode\
oft-MIEngine-Pid-240mli25.2gf' '--dbgE
Please input your name: Micheal
HELLO,Micheal!
PS E:\cpp> & 'c:\Users\Lancer\.vscode\
oft-MIEngine-Pid-wzmegaa0.uye' '--dbgE
Please input your name: Micheal
HELLO, Micheal!
PS E:\cpp> □
```

说明:

(1) C++源程序的扩展名为`.cpp`，而C源程序的扩展名是`.c`。

(2) 头文件:

- 示例中的`#include <iostream>`为C++的预处理指令, 它告诉C++编译器在程序中要包括C++ 标准输入输出流头文件。
- 从C继承的 “.h” 库仍然可用。在C源文件名前加字母c, 如 `#include <stdio.h>`在C++程序中可写为
`#include <cstdio.h>`。

一个简单的程序

编译预处理命令**#include**

1、**#**：预处理命令的标志。

2、**#include**：包含命令；

将文件**iostream**的内容包含到该命令所在的程序文件中,代替该指令。

3、其他预处理命令：**#define #undef #if #ifdef #ifndef #elif #endif**等

一个简单的程序

头文件（比如<iostream> “mylib.h”）

- 如果程序文件中引用到的函数、变量、常量、对象、数据类型等是由别的文件提供的，则必须在程序文件的开始部分，用**#include**命令把有关的头文件包含进来。
- 如果是 **#include <*.h>**，尖括号告诉预处理器在标准系统目录中查找该文件；
- 如果是 **#include "*.h"**，那么编译器是先从自定义的路径中寻找，如果找不到则再去自带的函数库中寻找；

```
#include <stdio.h>           ←查找系统目录  
#include "hot.h"             ←查找当前工作目录  
#include "/usr/biff/p.h"     ←查找/usr/biff目录
```


一个简单的程序

程序的注释：

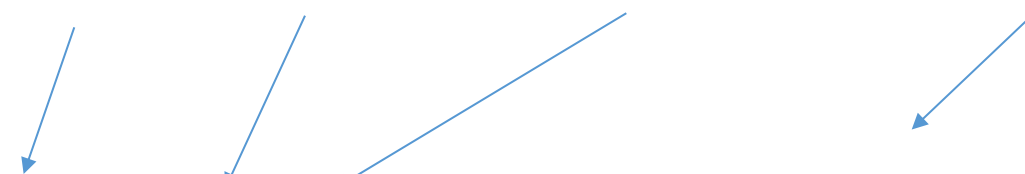
- 在C语言中：用 “**/***” 及 “***/**” 作为注释分界符号，
例如：`/* this is a test */`
 - C++：
 - 保留了C语言的注释方式
 - 增加了一种以 “**///**” 开头的单行注释方式，该注释以 “**///**” 开始，到行尾结束。
- `/*.....*/`方式的注释不能嵌套，但它可以嵌套`///`方式的注释。

一个简单的程序

函数（具有特定功能的程序模块）

1、定义格式：

返回类型 函数名（形式参数表）{函数体}



`int main ( ) {cout<< " 同学们, 你们好! ";}`

- (1) 类型修饰符：函数返回值的类型。
- (2) 函数名：非保留字的标识符。
- (3) 形式参数表：由一系列用逗号隔开的参数组成。
- (4) 函数体：包含在一对{ }中的语句序列。

一个简单的程序

主函数（**main**函数）

每个程序中至少要有一个函数，这个不可缺少（**一般情况下**）的函数就是主函数，约定主函数名为：**main**

- （1）程序由一个或多个函数组成。
- （2）如果程序中只有一个函数，则一定是**main**函数。
- （3）如果程序中有多个函数，则有且仅有一个为**main**函数。

一个简单的程序

main函数是程序的入口，程序是从**main**函数开始执行的。

main函数的返回值

☆ **int**: 有返回值。

(**0**表示程序正常结束

非**0**表示程序非正常结束)

输入/输出:

- C语言输入/输出:

scanf 和printf函数

- C++输入/输出:

- **cin**: 标准输入流对象
- **cout**: 标准输出流对象
- **>>**: 是输入运算符
- **<<**: 是输出运算符

cout<<数据; //表示把数据写到流对象cout上

cin>>变量; //表示从流对象cin读数据到变量中

- 流对象cin、cout及运算符<<、>>的定义, 均包含在文件iostream.h中。



一个简单的程序

C++语句

☆基本语句：以分号；作为结束标志。

☆复合语句：包含在{ }中的基本语句序列。

C++语言提供**名字空间** (namespace) 防止命名冲突。

1. 名字空间的定义

namespace 名字空间名称

```
{ ... ;
```

```
}
```

关于名字空间定义的说明：

- 定义名字空间以关键字namespace开头，名字空间名称是合法的用户自定义标识符。
- 以一对大括号括起该名字空间的开始和结束处，右大括号后面不加分号。
- 在名字空间以外声明或定义的任何实体，都可以出现在名字空间内。



名字空间中内容的使用

- 方法一：在需要使用名字空间中内容时用下面的形式：

名字空间名称::局部内容名

- 方法二：在使用该内容之前用：

using namespace 名字空间名称;

- 方法三：

using 名字空间名称::局部内容名;

关于名字空间的主要用法示例。

```
#include <iostream>
using namespace std;

//using声明使用一个完整的名字空间std
// C++中提供的名字空间std涵盖了所有标准C++的定义和声明

namespace one
//定义一个名字空间one，有M和inf

{ const int M=200;
  int inf=10;
} //后面不加分号
```



```
namespace two                                //定义一个名字空间two，有x和inf
{
    int x;
    int inf=-100 ;

}                                              //后面不加分号
```

```
using namespace one ;  
  
    //using声明使用一个完整的名字空间one  
  
int main(    )  
{    using two::x ;                //using声明仅使用two中的x  
    x=-100 ;                        //直接访问，相当于two::x=-100;  
    cout<<inf<<endl;              //直接使用one中所有成员  
    cout<<M<<endl;  
    two::inf*=2;  
  
    //使用名字空间名::局部内容名，操作未使用using声明的内容  
    cout<<two::inf<<endl;  
  
    //同样是two中的内容，但是访问方式不一样  
    cout<<x<<endl ;                //直接访问two中的x  
  
}
```

程序的调试运行

结果为：

```
10  
200  
-200  
-100  
请按任意键继续. . .
```



简单回顾

计算机系统概述

程序设计的基本概念与方法

程序设计语言及其发展

结构化程序设计方法与面向对象程序设计方法概述

C语言与C++语言：简单示例 名字空间等



程序设计风格

程序设计风格

- **目的：**提高程序的**可读性、可理解性和可修改性**，
以利于程序查错、测试及维护

要养成良好的程序设计风格！



程序设计风格

- 基本原则
 - 以简洁明了的方式编写C++程序
 - 缩排规则
 - 标识符命名规则（如匈牙利命名法、驼峰法、下划线法等）
 - 注释
 - 输出信息直观清晰，布局合理

程序设计风格：例子

```
#include<stdio.h>
```

```
int c(int a,int b){if(a>b){returna;}elseif(b>a){returnb;}else{return0;}}
```

```
#include <stdio.h>
```

```
// Compares two integers and returns the larger one. If they are equal, returns 0.
```

```
int max(int num1, int num2) {
```

```
    if (num1 > num2) {
```

```
        return num1; // Num1 is greater than num2
```

```
    } else if (num2 > num1) {
```

```
        return num2; // Num2 is greater than num1
```

```
    } else {
```

```
        return 0; // Numbers are equal
```

```
    }
```

```
}
```


- 除了遵循C++语言所规定的语法规则外，在命名标识符时建议遵循如下规则，将使程序的可读性和可维护性大大提高。这些规则包括：
- (1) 程序中避免用形式上只有细微差别的标识符名，如l0(数字0)与lo(字母o)；
- (2) 让标识符名反映其含义，使读者能见名知义；
- (3) 在一个程序中，注意标识符命名风格的一致性，对于符号常量名一般全部采用大写的标识符定义；
- (4) 对于类名，每个单词的开头字母一般用大写。如MyFirstApp。
- (5) 如果想用两个或更多的单词组成一个名称，通常的做法是用下划线字符、每个单词的首字母大写或从第二个单词开始将每个单词的首字母大写，如Student_name、StudentName或studentName，这三种方法依次称为下划线命名法、帕斯卡命名法和骆驼命名法，这几种形式都很容易将单词区分开，可以根据需要统一选择。

```
// Copyright 2008 Google Inc.
// License(BSD/GPL/...)
// Author: voidccc
// This is ...
```

```
#include "eventloop.h"
#include <sys/types.h>
#include <vector>
#include "base/basictypes.h"
#include "foo/public/bar.h"
```

```
using std::string
```

```
namespace mynamespace {
```

```
EventLoop::EventLoop()
```

```
    : _num_entries_(10),
      _num_completed_connections_(false) {}
```

```
    ...
```

```
}
```

```
ReturnType ClassName::ReallyLongFunctionName(const Type& par_name1,
                                                Type* par_name2) {}
```

```
bool retval = DoSomething(averyveryveryverylongargument1,
                           argument2, argument3);
```

```
if(condition) {}
```

```
for(int i = 0; i < kSomeNumber; ++i) {
    if (this_one_thing > this_other_thing &&
        a_third_thing == a_fourth_thing) {}
    // TODO(name@abc.com): xxx
}
```

```
}
```

```
} else {
    int j = g()
}
```

```
switch(var) {}
```

```
case 0: {}
    ...
break;
```

```
}
default: {}
    assert(false);
```

```
}
```

```
return x;
```

```
}
```

```
} // namespace mynamespace
```

版权
许可证, BSD/GPL/MIT/...

作者

文件内容简短说明

包含次序标准化避免隐藏依赖:

- 1 本类的声明(第一个包含本类h文件, 有效减少依赖)
- 2 C系统文件
- 3 C++系统文件
- 4 其他库头文件
- 5 本项目内头文件(避免使用UNIX文件路径"."和"..")

可以在整个cc文件和h文件的方法内使用using
禁止使用using namespace xx污染命名空间

多行初始化列表, ":"前4空格缩进, 以","结尾
多个变量折行对齐
单行初始化列表 Class::Class(): _var(xx) {}
构造函数中只进行那些没有实际意义的初始化

参数过多时,"结尾
每行一个变量对齐

参数过多时,"结尾

条件括号内无空格, (condition)左右1空格, if执行体2空格缩进

前置自增运算
条件变量过多时, 条件运算符 && 结尾
条件左对齐
临时方案使用TODO(大写)注释,
后面括号里加上你的大名、邮件地址等

大括号与else同行, else左右1空格
尽量使用初始化时声明

(var)左右各1空格
条件相对switch 2空格缩进
执行体相对switch 4空格缩进

若default永不执行可使用assert

返回值不需要加括号

命名空间结束注释

实现文件扩展名.cc
文件名全小写,可包含下划线或短横线

```
// Copyright 2008 Google Inc.
// License(BSD/GPL/...)
// Author: voidccc
// This is ...
```

```
#ifndef PROJECT_EVENTLOOP_H_
#define PROJECT_EVENTLOOP_H_
```

```
class Channel;
```

```
namespace mynamespace {
```

```
class EventLoop: public CallbackInterface {
public:
```

每一个限定符内, 声明顺序如下

- 1 typedefs和enums
- 2 常量
- 3 构造函数
- 4 析构函数
- 5 成员函数, 含静态数据成员
- 6 成员变量, 含静态成员变量

```
typedef vector<int> IntVector;
```

```
enum UriTableErrors {
    ERROR_OUT_OF_MEMORY = 0,
    ERROR_MALFORMED_INPUT,
```

```
};
```

```
explicit EventLoop(const int xx);
```

```
void Add(const std::string& input, Channel* output)
```

```
int num_entries() const { return num_entries_; }
```

```
void set_num_entries(int num_entries) { num_entries_ = num_entries; }
```

```
private:
```

```
DISALLOW_COPY_AND_ASSIGN(EventLoop);
```

```
const int kDaysInWeek = 7;
```

```
int num_entries_;
```

```
int num_completed_connections_;
```

```
Channel* channel_;
```

```
};
```

```
// namespace mynamespace
```

```
#endif // PROJECT_EVENTLOOP_H_
```

版权
许可证, BSD/GPL/MIT/...

作者

文件内容简短说明

头文件中尽量使用前置声明

STL类例外不使用前置声明, 使用 #include

命名空间全小写 顶头无空格 cc文件里提倡使用不具名命名空间

类名大写开头单词, 使用组合通常比使用继承更适宜
若用继承, 只用共有继承
另: 接口类命名以 "Interface" 结尾

Google C++ Style Guide

2空格缩进

枚举名同类名, 大写开头单词
枚举值2空格缩进, 全大写下划线

explicit修饰单参数构造函数, 防止隐式类型转换错误
若定义了成员变量无其他构造函数, 要定义一个默认构造函数

普通函数命名, 大写开头单词, 输入参数在前为const引用, 输出参数在后为指针
不为参数设置缺省值

存取函数命名, 取: 同变量名, 存: 值函数名为 set_varname
短小的存取函数可用内联

尽可能使用const

仅在需要拷贝对象时使用拷贝构造函数

不需要拷贝时在private里使用DISALLOW_COPY_AND_ASSIGN宏
变量用描述性名称, 不要节约空间, 让别人理解你的代码更重要
const 变量为k开头, 后跟大写开头单词
变量命名: 全小写, 有意义的单词和下划线
类成员变量下划线结尾

头文件中只用了指针/引用,
则前向声明而非引入头文件

上下少空行, 每屏代码越多越好

左右小于80

保护宏结尾加注释

复盘一下：面向过程，面向对象？



面向过程

- 定义函数

- 函数1
- 函数2
- 函数3
- 函数4

- 定义数据

- 数据1
- 数据2
- 数据3
- 数据4

命名冲突
代码重复

- 各种函数、数据的操作



以计算器小程序为例

- 想象我们要做一个计算器的小程序。加减乘除函数要确保处理的是合理的数据，因此都要写对参数进行检测的代码。我们可以将这个“检测代码”独立出来，因此有如下函数：
- 检测，整数加，整数减，整数乘，整数除，复数加，复数减，复数乘，复数除，小数加.....

以计算器小程序为例：类

整数 {
 检测
 加
 减
 乘
 除
}

复数 {
 检测
 加
 减
 乘
 除
}

小数 {
 检测
 加
 减
 乘
 除
}

.....



类

以计算器小程序为例：继承

数 {
 检测
 加
 减
 乘
 除
}

整数 {
 沿用上面数的设计
}

小数 {
 沿用上面数的设计
}

.....

有了类的继承，还能继续改进吗？

```
小数类 {  
    加  
    减  
    乘  
    除  
}
```

如果我想一个小数计算类有四舍五入功能，另一个不要，难道需要写两个类吗？



有了类的继承，还能继续改进吗？

小数类 {

标识变量：是否四舍五入

标识变量：是否限定小数点后位数

构造函数（设置上面的标识）

加（会根据上面两个标识变量输出不同结果）

减（会根据上面两个标识变量输出不同结果）

乘（会根据上面两个标识变量输出不同结果）

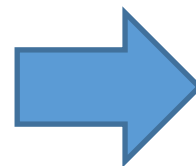
除（会根据上面两个标识变量输出不同结果）

}

以计算器小程序为例：模板

单整数类 {
 单整数加
 单整数减
 单整数乘
 单整数除
}

双整数类 {
 双整数加
 双整数减
 双整数乘
 双整数除
}



<变量类型>整数 {
 <变量类型>加
 <变量类型>减
 <变量类型>乘
 <变量类型>除
}

以计算器小程序为例：重载

加 “+”：能否对其赋予新含义，使其一名多用？

能实现普通加法，能否实现复数的加法？

原来：写函数， $c3=c1.\text{complex_add}(c2)$ 或 $c3 = \text{complex_add}(c1, c2)$

重载： $c3=c1+c2$

例子回顾：计算器小程序

- 面向过程：函数若干，数据若干（命名冲突，代码重复）
- 提取共性函数（检测）
- 归“类”（整数、复数、小数等）
- 继续提取/抽象，得到父类、子类，为继承关系
- 模板功能
- 实例，对象，构造函数.....
- 重载？多态？

回顾一下：

- 程序是对计算任务的**处理对象**和**处理规则**的描述，它规定了计算机执行的动作和动作的顺序

- 组成

- 数据

程序处理的对象，具有**数据结构**

- 操作

处理数据的操作步骤，即**算法**

程序 = 数据结构 + 算法

Niklaus Wirth

面向对象的语境中有什么变化呢？

程序 = 数据结构 + 算法



对象 = 数据结构 + 算法

程序 = (对象+对象+对象...) + 消息



福州大学
FUZHOU UNIVERSITY

教学内容

- 绪论
- C++ 基础知识
 - 数据类型、表达式
 - 输入与输出
 - 控制结构
 - 函数
 - 数组
 - 指针
 - 结构/联合/枚举
 - 链表
- 面向对象的程序设计
 - OOP的基本概念
 - 类与对象
 - 运算符重载
 - 继承与派生
 - 多态性与虚函数
 - 文件和流
 - 其它工具



古人的智慧？

- 司马迁的《史记》
- 面向过程→编年体， 面向对象→纪传体

古人的智慧？

- **编年体**就是用**面向过程**的思想叙事
 - 面向过程的思想就是，我不管这事情主角是谁，我就按事情的发展**过程**叙述。这个形式里强调描述“过程”。就好比编年体里，我会把1000年里每一年有什么事情发生罗列在一起，让你看完之后，对事情的**过程**很清晰，但每个人物都干过啥，理不清了。

古人的智慧？

- **纪传体**就是用**面向对象**的思想叙事

- 面向对象思想是，我强调一件事情的“主角”是谁，所有和他相关的内容都封装在一起。就好比纪传体，一个人物单独封装成一个章节。这个封装里面，主角（**对象**）非常清晰，一切事情都是围绕它的。这个做法的好处是，如果你想查一查张三是否做过某某事，你可以只查张三那章内容就好了。

面向过程

- 优点

- 流程化使得编程任务明确，在开发之前基本考虑了实现方式和最终结果，具体步骤清楚，便于节点分析。
- 效率高，面向过程强调代码的短小精悍，善于结合数据结构来开发高效率的程序。、

- 缺点

- 需要深入的思考，耗费精力，代码重用性低，扩展能力差，后期维护难度比较大。

面向对象

• 优点

- 结构清晰，程序是模块化和结构化，更加符合人类的思维方式；
- 易扩展，代码重用率高，可继承，可覆盖，可以设计出低耦合的系统；
- 易维护，系统低耦合的特点有利于减少程序的后期维护工作量。

• 缺点

- 开销大，当要修改对象内部时，对象的属性不允许外部直接存取，所以要增加许多没有其他意义、只负责读或写的行为。这会为编程工作增加负担，增加运行开销，并且使程序显得臃肿。
- 性能低，由于面向更高的逻辑抽象层，使得面向对象在实现的时候，不得不做出性能上面的牺牲，计算时间和空间存储大小都开销很大。

学习目的检测

- 了解计算机系统、程序设计和程序设计语言的基本概念
- 了解结构化程序设计和面向对象程序设计方法
- 掌握简单C++程序的结构
- 明确程序设计风格的意义





Thank you !



福州大学
FUZHOU UNIVERSITY



福州大学
FUZHOU UNIVERSITY



福州大学
FUZHOU UNIVERSITY